배틀아레나 외주 서버 인수 검토 보고서

과목: 게임 서버 프로그래밍

과제명: 외주 서버 인수 검토(TICKET-099)

학번: 24240907

이름: 정채원

목차

검토 개요 3

Part A. 외주 서버 결함 분석 4

A-1. TCP 분할 수신 4

A-2. 비정상 패킷과 서버 종료 7

A-3. UDP 중복 시퀀스 10

A-4. UDP 수신 데이터 길이 13

A-5. UDP 송신 주소 검증 15

A-6. 클라이언트 UDP 바인딩 17

A-7. 퇴장 알림 20

Part B. 동시 접속 부하 측정 22

B-1. 측정 환경과 방법 22

B-2. 수정 전·후 측정 결과 23

B-3. 성능 저하가 시작된 구간 23

B-4. 성능 저하 원인 24

B-5. 수정 내용과 재측정 결과 25

Part C. 업체 검토서 판정 27

C-1. 업체 의견 10 건 판정 27

C-2. 의견 3 번 await 추가 실험 28

C-3. 의견 10 번 부하 측정 검증 31

Part D. 작업 과정 기록 33

D-1. Git 작업 기록 33

D-2. 시행착오 ① UDP 수신 문제 35

D-3. 시행착오 ② 동시 접속 실패 37

최종 판단 39

증거 자료 목록 40

검토 개요

외주 업체가 납품한 배틀아레나 서버를 직접 실행하고, 함께 제공된 TestClient 와 LoadTester 로 기능과 동시 접속 성능을 확인했다. 서버는 TCP 로 로그인과 채팅을 처리하고 UDP 로 위치를 주고받는 구조다. 내 학번을 설정한 뒤 TCP 12066 번, UDP 12067 번 포트로 테스트했다.

원본은 접속 자체가 되는 경우에도 분할 패킷, 비정상 패킷, UDP 처리, 퇴장 알림에서 문제가 있었다. 동시 접속 시험에서도 50 명부터 실패가 크게 늘었다. 확인된 문제 7 건을 수정한 뒤 같은 테스트를 다시 실행했고, 수정 전후 결과를 비교했다. 아래 내용은 실제 실행한 범위와 측정 결과를 기준으로 정리했다.

Part A. 외주 서버 결함 분석

A-1. TCP 패킷을 나누어 보내면 정상적으로 읽지 못하는 문제

재현 방법

1. 원본 OutsourcedServer 를 실행한다.

2. LoadTester 에서 2 번 분할 전송을 선택한다. 이 테스트는 채팅 패킷의 앞부분을 보내고 50ms 후 나머지 부분을 보낸 다음, 정상 패킷을 하나 더 보낸다.

3. 서버가 전송한 채팅 내용을 LoadTester 결과 화면에서 확인한다. 수정 후에도 같은 방법으로 시험한다.

수정 전 결과

분할한 채팅 패킷과 그다음 정상 패킷이 기대한 대로 돌아오지 않았다. 서버 오류 화면도 함께 남겼다.

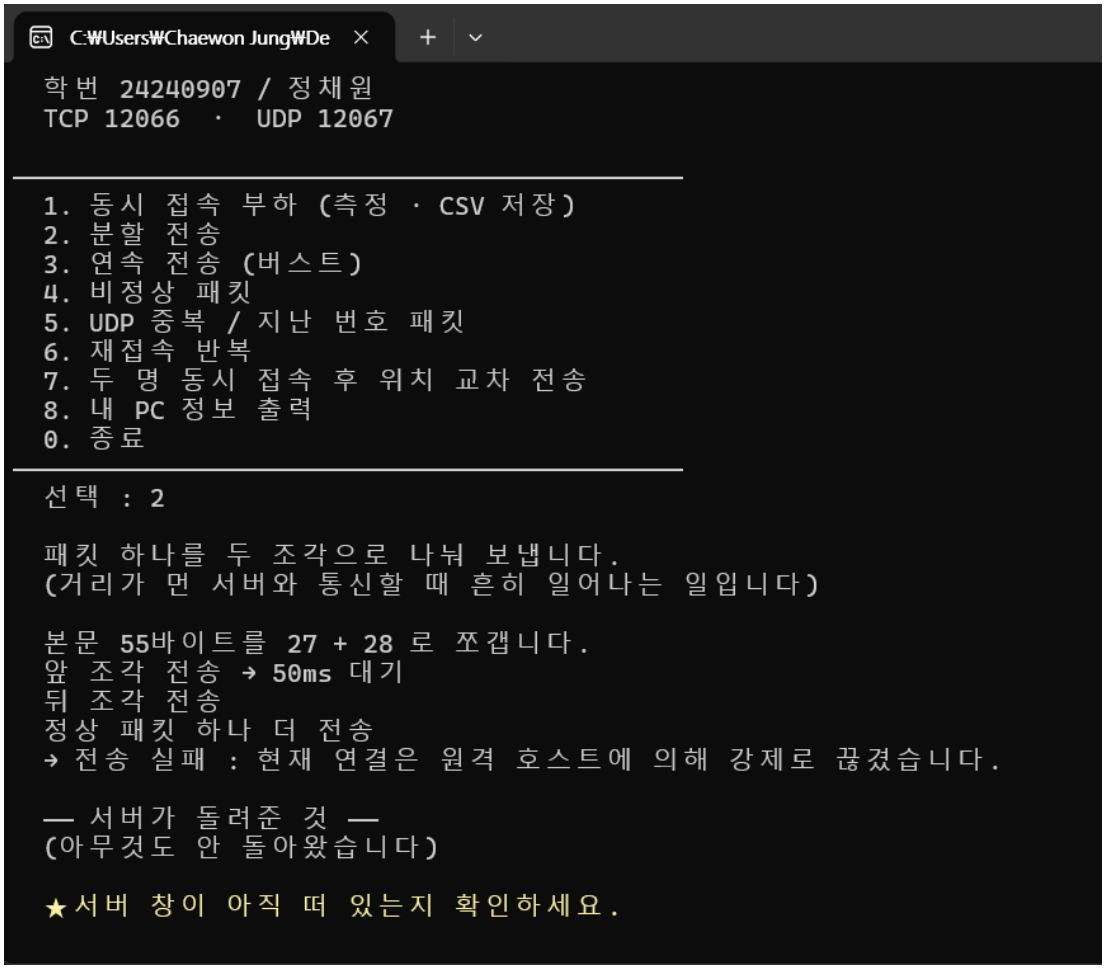


그림 1. A-1 수정 전 — 분할 전송 후 응답 없음

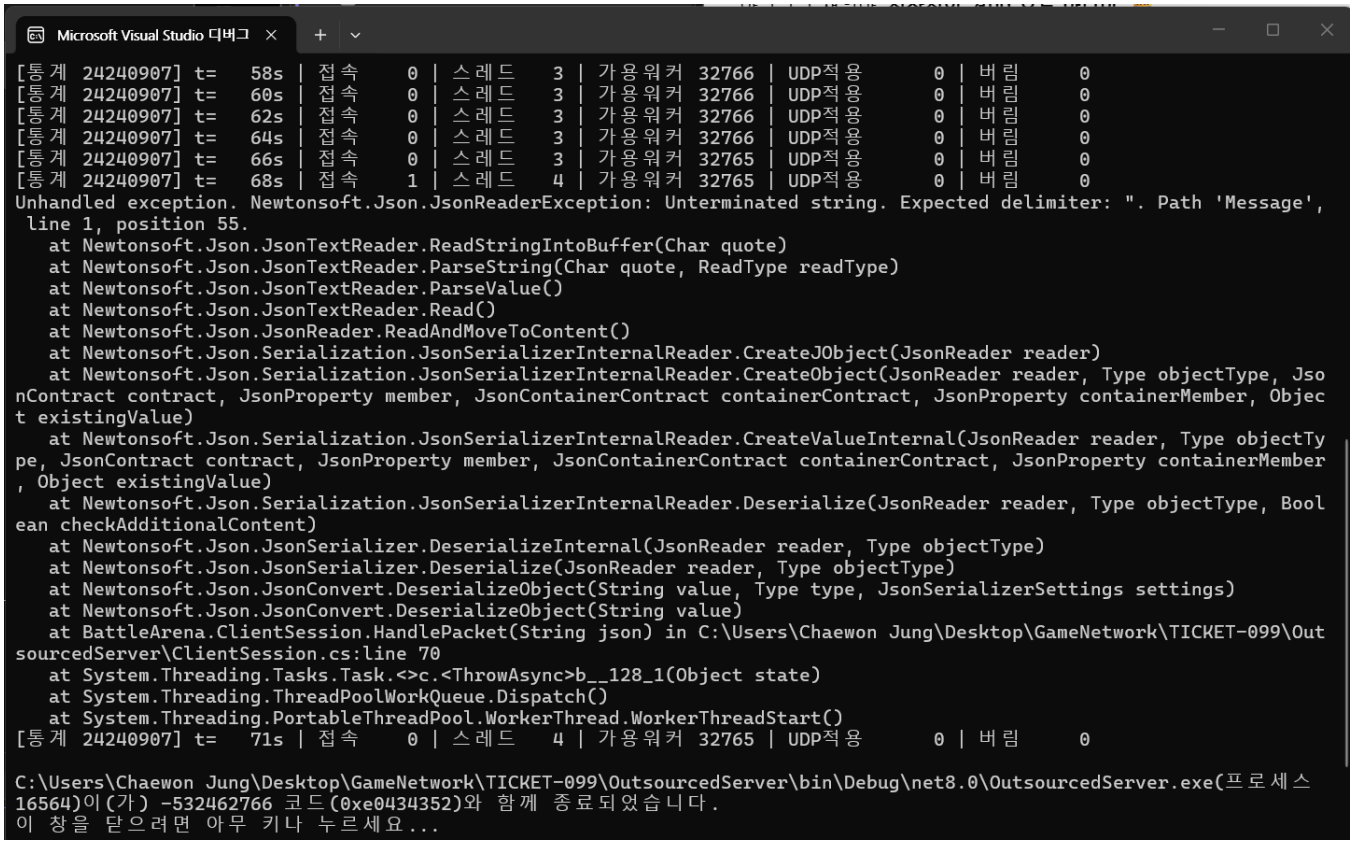


그림 2. A-1 수정 전 서버 오류 — 분할 전송 후 JSON 읽기 오류

원인

`OutsourcedServer/PacketHelper.cs` 원본 51 행과 63 행은 `socket.Receive()`를 한 번만 호출한다. TCP 의 `Receive()`는 요청한 길이만큼 항상 채워 준다는 보장이 없으므로, 헤더 4 바이트 또는 본문이 일부만 들어왔을 때 불완전한 데이터를 처리할 수 있다. 여기에 `Task.Run` 으로 블로킹 수신을 감싼 방식도 스레드 점유 문제를 만든다.

수정 전 핵심 코드 (`PacketHelper.cs`, 51 행·63 행):

```
int n = await Task.Run(() => socket.Receive(header, 0, 4, SocketFlags.None));
await Task.Run(() => socket.Receive(body, 0, bodyLength, SocketFlags.None));
```

수정 후 핵심 코드 (`PacketHelper.cs`, 50~81 행):

```
public static async Task<string?> ReceiveAsync(Socket socket)
{
    byte[] header = new byte[4];
    if (!await ReadAsync(socket, header, 4)) return null;

    int bodyLength = BitConverter.ToInt32(header, 0);
    if (bodyLength <= 0 || bodyLength > MAX_BODY) return null;
    byte[] body = new byte[bodyLength];
    if (!await ReadAsync(socket, body, bodyLength)) return null;

    return Encoding.UTF8.GetString(body, 0, bodyLength);
}
```

```
private static async Task<bool> ReadAsync(Socket socket, byte[] buffer, int size)
{
    int read = 0;
    while (read < size)
    {
        int n = await socket.ReceiveAsync(
            new ArraySegment<byte>(buffer, read, size - read), SocketFlags.None);
        if (n == 0)
            return false;
        read += n;
    }

    return true;
}
```

수정 후 확인

분할 전송을 다시 실행했을 때 분할된 채팅 메시지와 이어서 보낸 정상 메시지가 모두 돌아왔다. TCP 를 여러 번 나누어 받아도 필요한 바이트 수를 채울 때까지 기다리도록 바뀐 결과다.

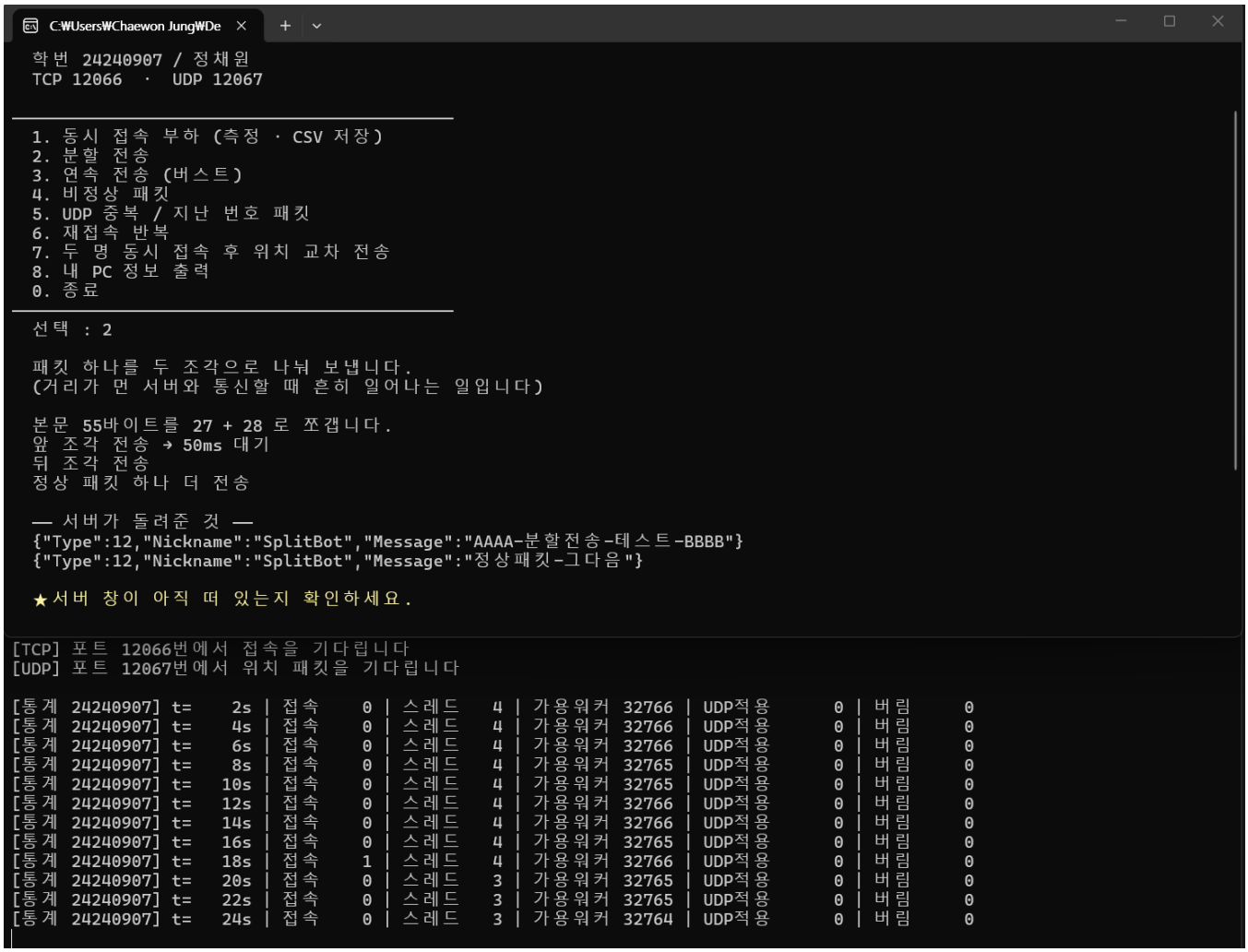


그림 3. A-1 수정 후 — 분할 메시지와 다음 정상 메시지 수신

증거: A_버그1_수정전_서버오류.png, A_버그1_수정전_테스트결과.png, A_버그1_수정후.png

A-2. 비정상 패킷 때문에 서버가 종료되는 문제

재현 방법

1. 원본 서버를 실행한다.
2. LoadTester 에서 4 번 비정상 패킷을 선택해 `{"Nickname":"없음"}`처럼 `Type` 필드가 없는 JSON 을 보낸다.
3. 이어서 정상 채팅 패킷을 보내 서버가 계속 동작하는지 확인한다. 수정 후에도 동일한 테스트를 진행한다.

수정 전 결과

서버에서 `RuntimeBinderException` 이 발생하며 프로그램이 종료됐다. 오류 메시지에는 `null` 을 `int` 로 변환할 수 없다고 표시됐다.

```
  1. 동시 접속 부하 (측정 · CSV 저장)
  2. 분할 전송
  3. 연속 전송 (버스트)
  4. 비정상 패킷
  5. UDP 중복 / 지난 번호 패킷
  6. 재접속 반복
  7. 두 명 동시 접속 후 위치 교차 전송
  8. 내 PC 정보 출력
  0. 종료
  선택 : 4

  형식이 깨진 패킷을 보냅니다.
  (인터넷에 서버를 열면 실제로 이런 트래픽이 들어옵니다)

  1) Type 필드가 없는 JSON 전송
  2) 그 다음 정상 패킷 전송 – 서버가 살아있는지 확인
     → {"Type":12,"Nickname":"BadBot","Message":"아직 살아있나요?"}

  ★서버 창을 지금 바로 확인하세요. 아직 떠 있습니까?
  ★떠 있지 않다면, 서버 콘솔에 마지막으로 뭐가 찍혔는지 캡처하세요.

  학번 24240907 / 정채원
  TCP 12066  ·  UDP 12067

[TCP] 포트 12066번에서 접속을 기다립니다
[UDP] 포트 12067번에서 위치 패킷을 기다립니다

[통계 24240907] t=    2s | 접속    0 | 스레드   4 | 가용워커 32766 | UDP적용      0 | 버림     0
[통계 24240907] t=    4s | 접속    0 | 스레드   4 | 가용워커 32766 | UDP적용      0 | 버림     0
[통계 24240907] t=    6s | 접속    0 | 스레드   4 | 가용워커 32766 | UDP적용      0 | 버림     0
[통계 24240907] t=    8s | 접속    0 | 스레드   4 | 가용워커 32765 | UDP적용      0 | 버림     0
[통계 24240907] t=   10s | 접속    0 | 스레드   4 | 가용워커 32766 | UDP적용      0 | 버림     0
Unhandled exception. Microsoft.CSharp.RuntimeBinder.RuntimeBinderException: Cannot convert null to 'int' because it is a
 non-nullable value type
   at CallSite.Target(Closure, CallSite, Object)
   at System.Dynamic.UpdateDelegates.UpdateAndExecute1[T0,TRet](CallSite site, T0 arg0)
   at CallSite.Target(Closure, CallSite, Object)
   at BattleArena.ClientSession.HandlePacket(String json) in C:\Users\Chaewon Jung\Desktop\GameNetwork\TICKET-099\Outsou
rcedServer\ClientSession.cs:line 73
   at System.Threading.Tasks.Task.<>c.<ThrowAsync>b__128_1(Object state)
   at System.Threading.ThreadPoolWorkQueue.Dispatch()
   at System.Threading.PortableThreadPool.WorkerThread.WorkerThreadStart()
[통계 24240907] t=   13s | 접속    1 | 스레드   5 | 가용워커 32762 | UDP적용      0 | 버림     0

C:\Users\Chaewon Jung\Desktop\GameNetwork\TICKET-099\OutsourcedServer\bin\Debug\net8.0\OutsourcedServer.exe(프로세스 880
4)이(가) -532462766 코드(0xe0434352)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

그림 4. A-2 수정 전 — Type 누락 패킷 처리 중 서버 종료

원인

OutsourcedServer/ClientSession.cs 원본 68 행의 패킷 처리 함수가 async void 였고, 71~73 행 부근에서 Type 이 있는지 확인하지 않은 채 (int)packet.Type 으로 변환했다. Type 이 빠지면 예외가 발생하고, async void 의 예외는 호출 측 수신 루프에서 정상적으로 기다려 처리하기 어렵다.

수정 전 핵심 코드 (ClientSession.cs, 50 행·68~73 행):

```
HandlePacket(json);

private async void HandlePacket(string json)
{
    dynamic? packet = JsonConvert.DeserializeObject(json);
    if (packet == null) return;
    int type = (int)packet.Type;
```

수정 후 핵심 코드 (ClientSession.cs, 50 행·68~80 행):

```
await HandlePacketAsync(json);

private async Task HandlePacketAsync(string json)
{
    dynamic? packet = JsonConvert.DeserializeObject(json);
    if (packet == null)
        return;

    if (packet.Type == null)
    {
        Console.WriteLine("[경고] Type 없는 패킷");
        return;
    }

    int type = (int)packet.Type;
```

수정 후 확인

같은 비정상 JSON 을 보냈을 때 Type 없는 패킷 경고가 출력됐고 서버는 종료되지 않았다. 패킷 처리 함수를 Task 반환으로 바꿔 호출 측에서 await 하도록 한 것도 예외 처리를 안정적으로 연결한 수정이다.

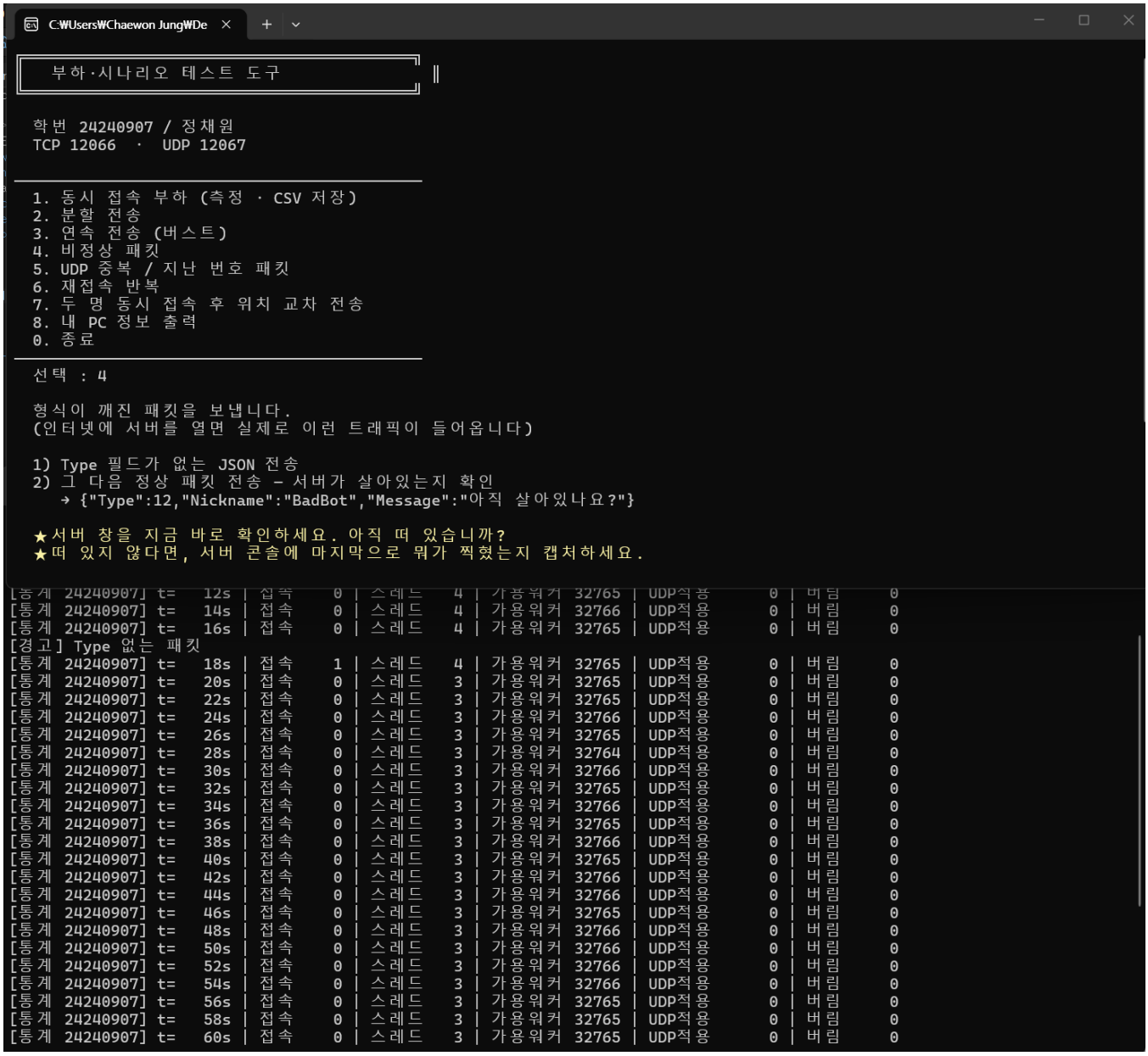


그림 5. A-2 수정 후 — Type 없는 패킷 경고와 서버 동작 유지

증거: A_버그 2_수정전.png, A_버그 2_수정후.png

A-3. UDP 중복 시퀀스 번호를 걸러내지 못하는 문제

재현 방법

1. 서버를 실행하고 LoadTester 에서 5 번 UDP 중복·지난 번호 패킷을 선택한다.

2. 시퀀스 1~20 을 정상 전송하고, 11~20 을 한 번 더 보낸 다음, 지난 번호 5~9 를 전송한다.

3. 서버 [통계]의 UDP 적용과 버림 값을 확인하고, 수정 전후 결과를 비교한다.

수정 전 결과

정상적으로 적용해야 할 패킷은 20 개, 버려야 할 패킷은 중복 10 개와 지난 번호 5 개로 총 15 개였다. 그런데 원본 서버는 적용 21 개, 버림 14 개로 집계했다.

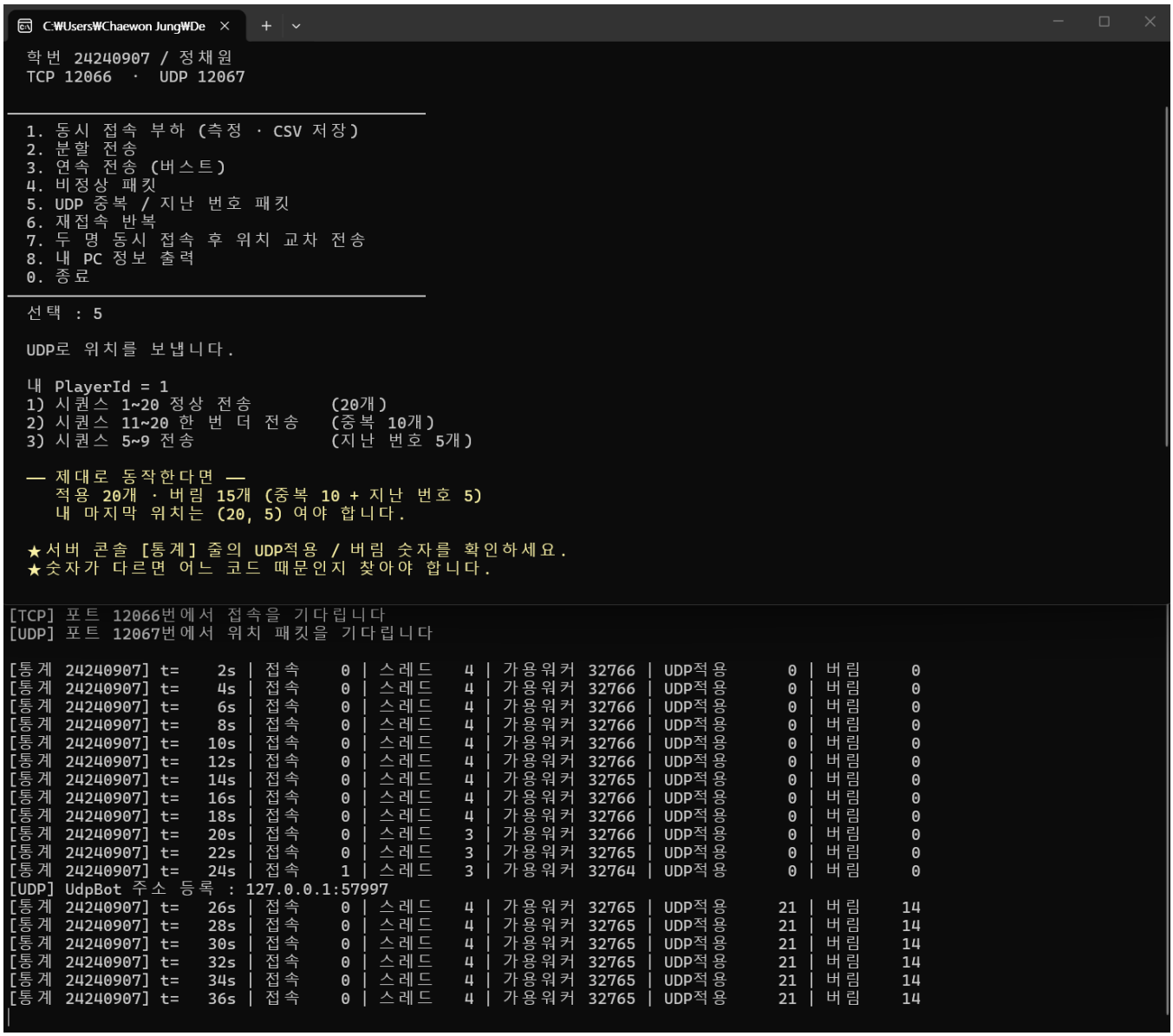


그림 6. A-3 수정 전 — UDP 적용 21 개, 버림 14 개

원인

`OutsourcedServer/UdpServer.cs` 원본 129 행은 `seq < session.LastSeq` 만 검사했다. 현재 마지막 번호와 같은 시퀀스 번호는 중복인데도 통과시킬 수 있었다.

수정 전 (UdpServer.cs, 129 행):

```
if (seq < session.LastSeq)
```

수정 후 (UdpServer.cs, 135 행):

```
if (seq <= session.LastSeq)
```

수정 후 확인

같은 35 개 패킷을 전송한 결과 적용 20 개, 버림 15 개가 나왔다. 마지막 위치도 `(20, 5)`로 확인했다. 지난 패킷뿐 아니라 같은 번호의 중복 패킷도 버리도록 바뀌었다.

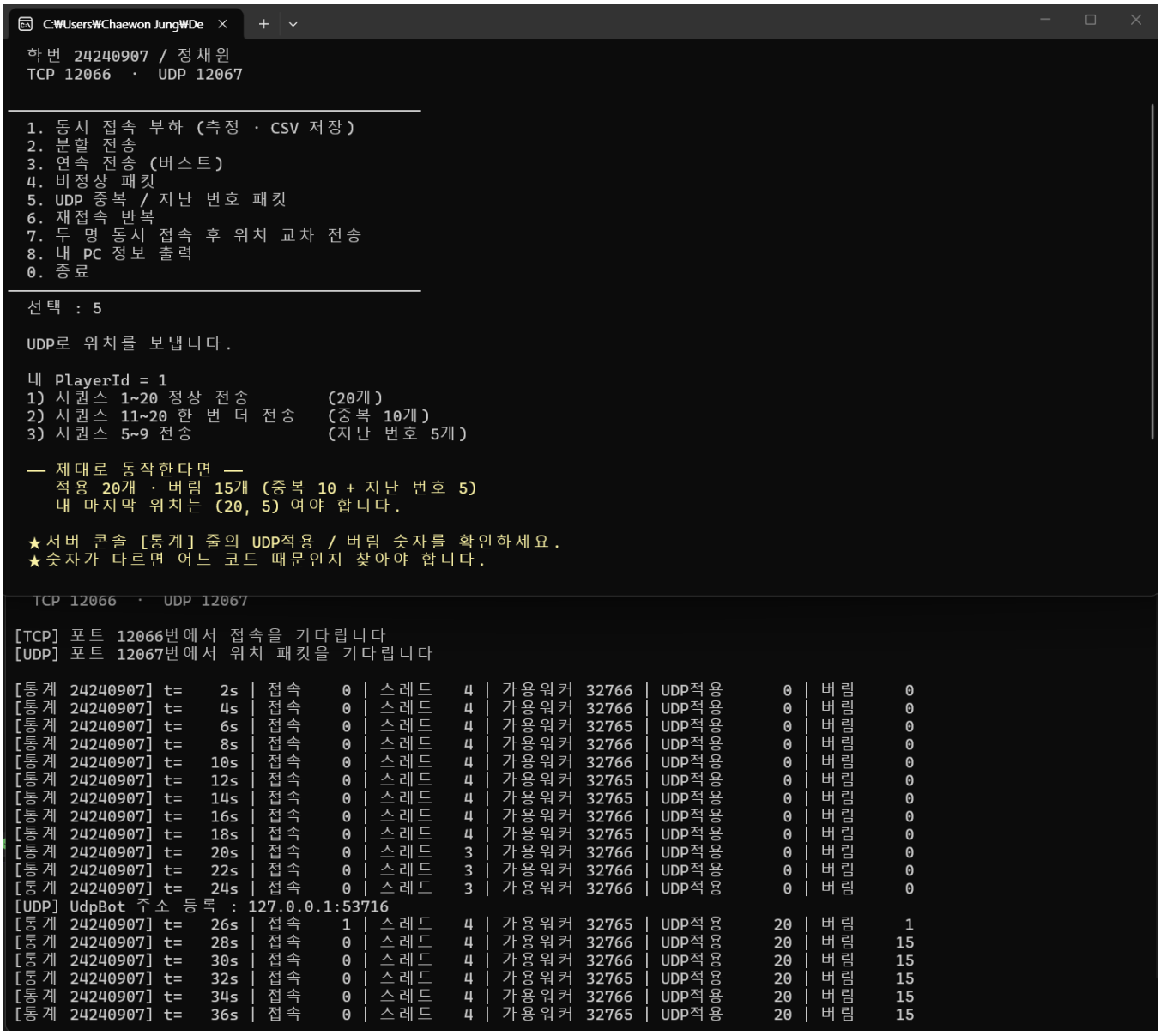


그림 7. A-3 수정 후 — UDP 적용 20 개, 버림 15 개

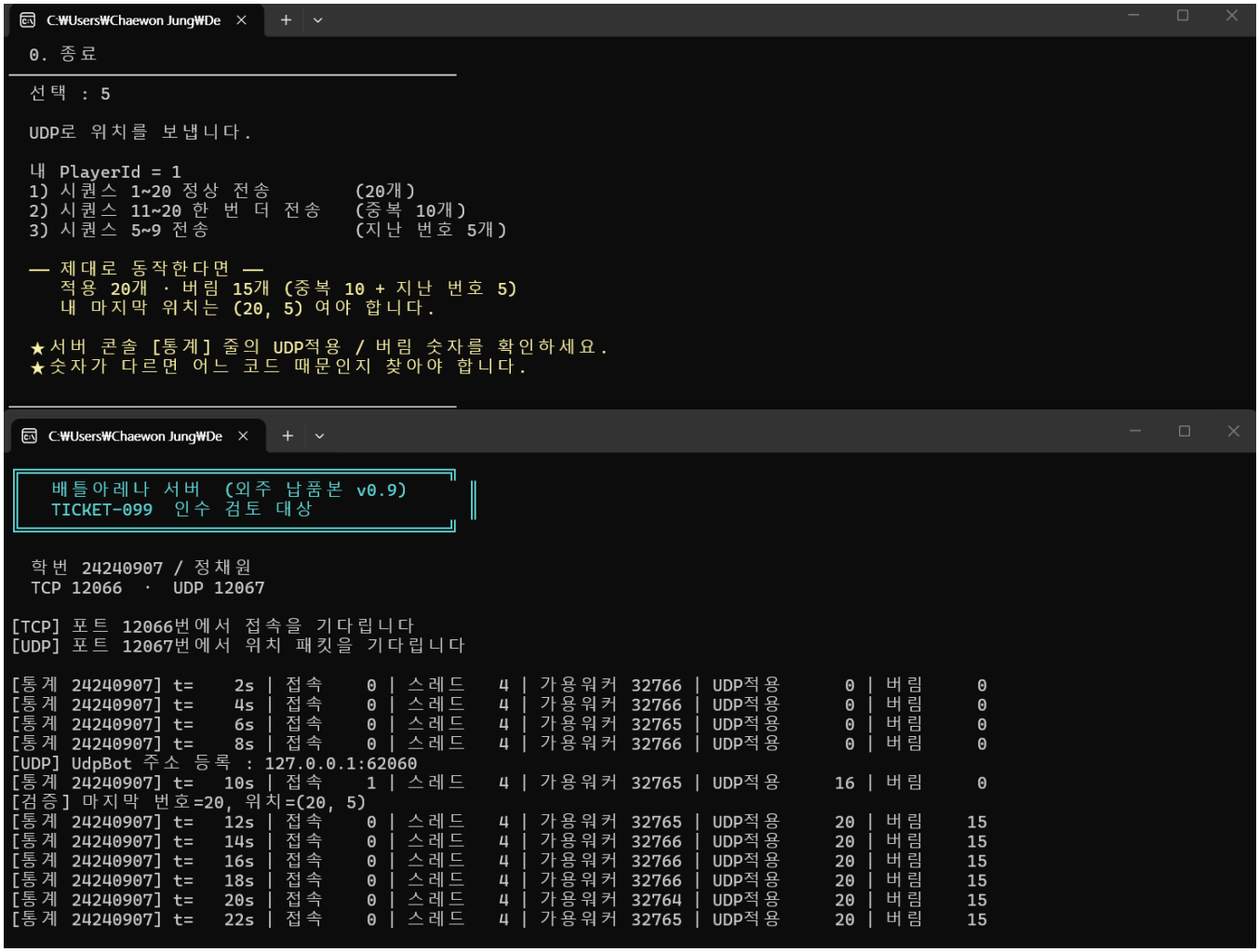


그림 8. A-3 수정 후 추가 확인 — 적용 20 개, 버림 15 개, 마지막 위치 (20, 5)

증거: A_버그 3_수정전.png, A_버그 3_수정후.png, A_버그 3_마지막위치확인.png

A-4. UDP 수신 버퍼의 남은 데이터까지 문자열로 변환하는 문제

재현 방법

1. 서버를 실행하고 LoadTester 에서 6 번 재접속 반복을 선택한다.

2. 접속, 위치 전송, 종료 과정을 5 회 반복하면서 각 회차에 수신한 STATE 문자열을 확인한다.

3. 동일한 테스트를 수정 전후에 실행해 위치 값에 불필요한 숫자가 붙는지 확인한다.

수정 전 결과

받은 STATE 에 STATE|2,ReBot2,6,110, STATE|3,ReBot3,9,210 처럼 위치 값에 이전 수신 데이터가 남은 것으로 보이는 결과가 나타났다. 모든 회차가 같은 형태로 잘못된 것은 아니었다.

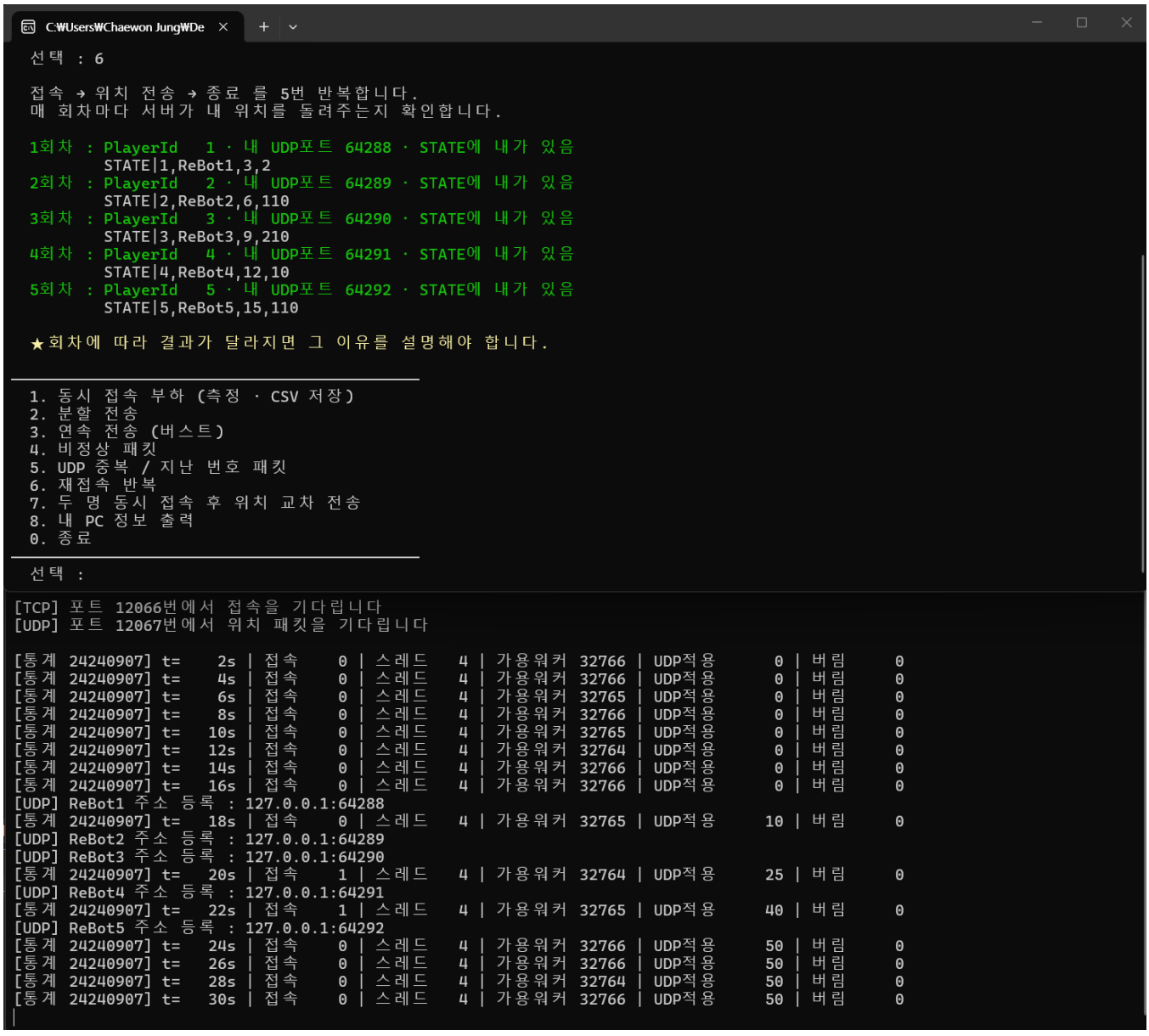


그림 9. A-4 수정 전 — 재접속 시 STATE 위치 값에 남은 데이터 포함

원인

OutsourcedServer/UdpServer.cs 원본 80 행에서는 UDP 수신 버퍼 전체를 문자열로 변환했다. 실제로 받은 데이터의 길이가 버퍼 크기보다 짧으면 뒤에 남아 있는 바이트까지 함께 해석할 수 있다.

수정 전 (UdpServer.cs, 80 행):

```
string msg = Encoding.UTF8.GetString(buffer);
```

수정 후 (UdpServer.cs, 80 행):

```
string msg = Encoding.UTF8.GetString(buffer, 0, result.ReceivedBytes);
```

수정 후 확인

재접속 반복 테스트에서 이전처럼 110, 210 등의 값이 섞이지 않았고, STATE 에 수신한 위치 값이 정상적인 형식으로 표시됐다. UDP 메시지를 처리할 때는 버퍼 전체가 아니라 ReceivedBytes 만큼만 읽어야 한다는 것을 확인했다.

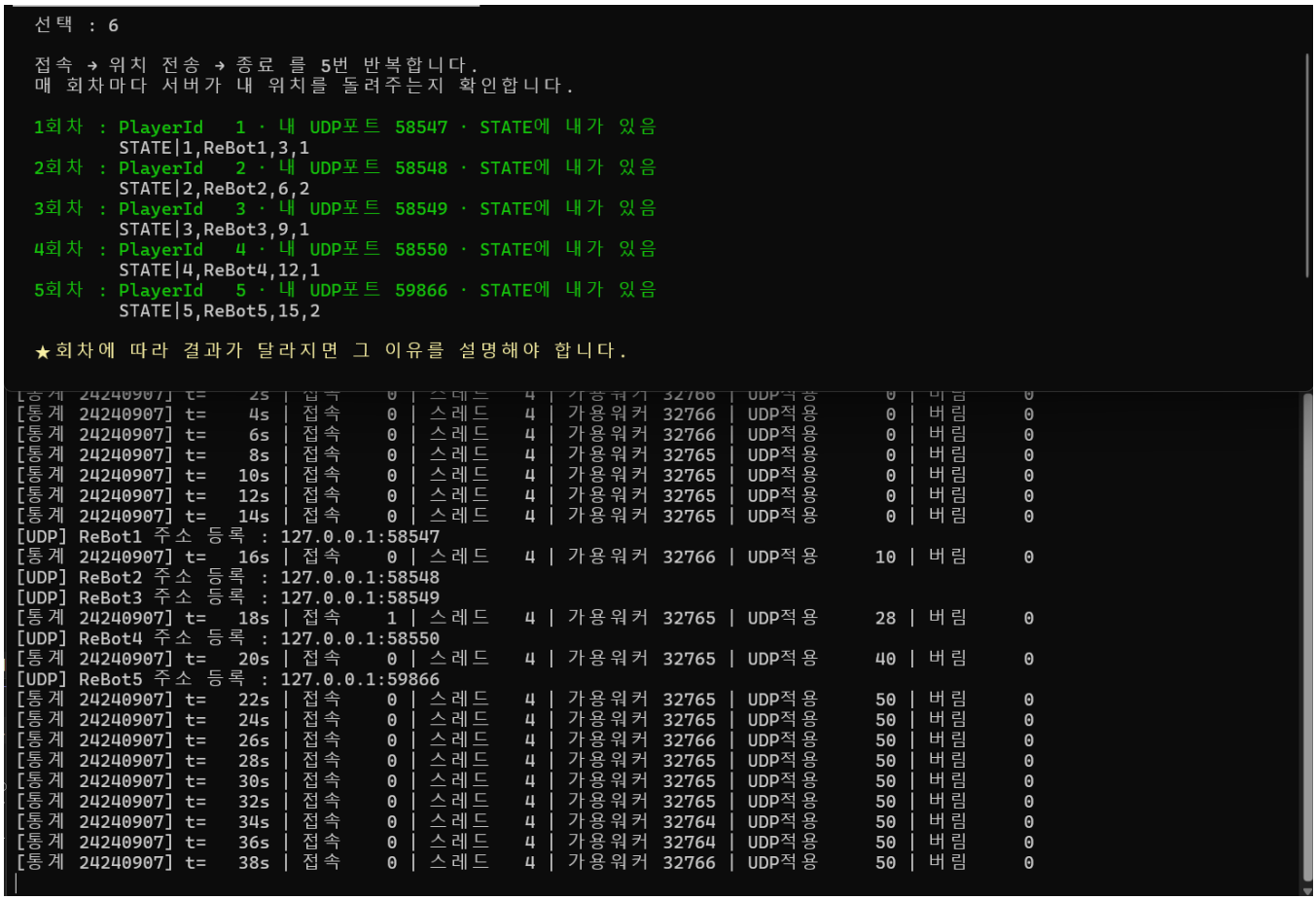


그림 10. A-4 수정 후 — 수신 길이만큼 해석한 STATE 결과

증거: A_버그 4_수정전.png, A_버그 4_수정후.png

A-5. 다른 UDP 소켓이 같은 플레이어 ID 로 보낸 패킷을 차단하지 못하는 문제

재현 방법

1. 서버를 실행하고 LoadTester 에서 7 번 두 명 동시 접속 후 위치 교차 전송을 선택한다.
2. BotA 와 BotB 가 각자의 UDP 소켓으로 위치를 보낸 뒤, BotB 가 BotA 의 ID 를 적은 `MOVE|1|9999|30|15` 형식의 패킷을 다시 보낸다(실제 ID 는 테스트 때 부여된 값을 사용한다).
3. 서버의 UDP 적용 횟수와 경고 메시지를 수정 전후에 비교한다.

수정 전 결과

BotB 가 BotA 의 ID 를 사용한 패킷까지 서버의 UDP 적용 횟수에 포함되어 3 개가 적용됐다. 다만 캡처에 출력된 BotA 의 STATE 좌표는 `(5, 5)`였으므로, 이 화면만으로 실제 좌표가 `(30, 15)`로 바뀌었다고 단정할 수는 없다.

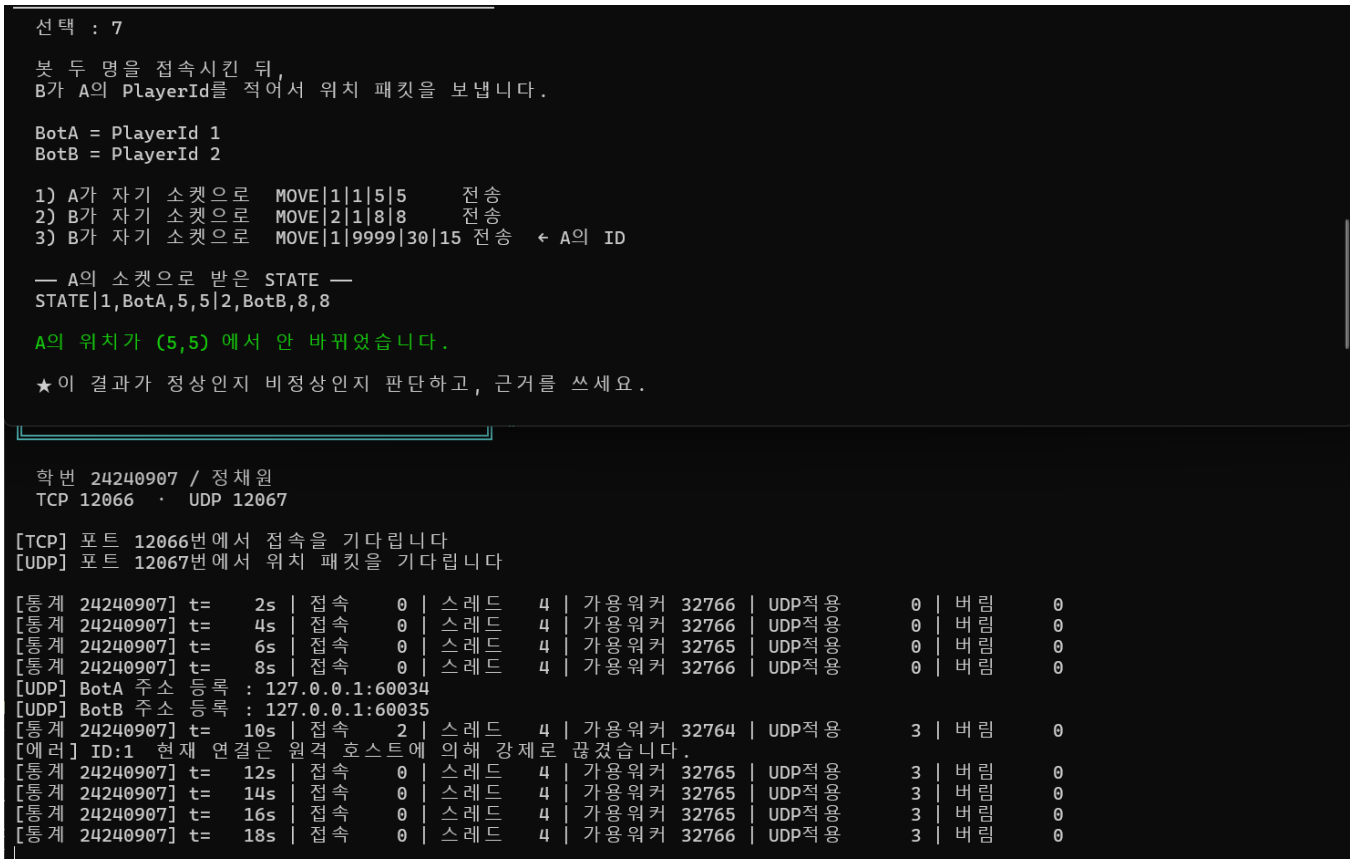


그림 11. A-5 수정 전 — 다른 주소의 패킷까지 UDP 적용 3 개로 집계

원인

`OutsourcedServer/UdpServer.cs` 원본 120~125 행에서는 첫 UDP 패킷으로 주소를 등록했지만, 이후 패킷의 송신 주소가 등록한 주소와 같은지 확인하지 않았다. 서버가 패킷의 플레이어 ID 만 믿고 처리할 여지가 있었다.

수정 전 핵심 코드 (UdpServer.cs, 120~124 행):

```
if (session.UdpEndPoint == null)
{
    session.UdpEndPoint = from;
    Console.WriteLine($"[UDP] {session.Nickname} 주소 등록 : {from}");
}
```

수정 후 추가 코드 (UdpServer.cs, 126~130 행):

```
else if (!session.UdpEndPoint.Equals(from))
{
    Console.WriteLine($"[경고] ID:{playerId} UDP 주소가 불일치 합니다.");
    return;
}
```

수정 후 확인

같은 교차 전송을 했을 때 서버에 주소 불일치 경고가 출력됐고 UDP 적용은 2 개로 유지됐다. 등록한 주소가 아닌 곳에서 보낸 패킷을 적용하기 전에 차단한 것이다. 참고로 이 차단은 버림 카운터를 올리는 분기와는 별개라서 버림 수치가 증가하지는 않는다.

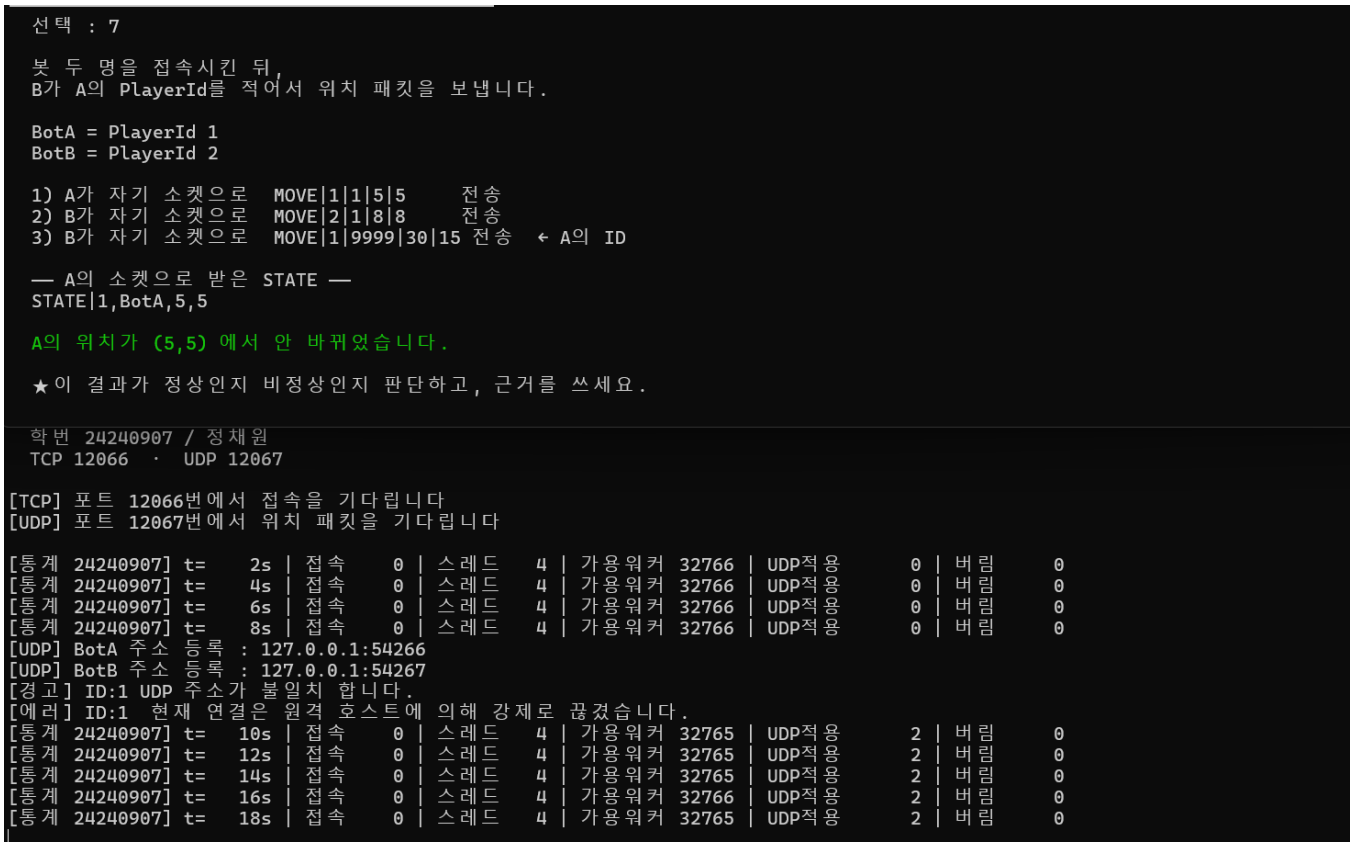


그림 12. A-5 수정 후 — 주소 불일치 경고, UDP 적용 2 개 유지

증거: A_버그 5_수정전.png, A_버그 5_수정후.png

A-6. 테스트 클라이언트의 UDP 수신 소켓 바인딩 누락

재현 방법

1. 원본 서버와 TestClient 를 실행해 TCP 로 접속한다.

2. 클라이언트의 위치 수신 기능을 실행하고 UDP 수신 관련 오류가 뜨는지 확인한다.

3. TestClient 를 수정한 뒤 다시 접속해 같은 동작을 실행한다.

수정 전 결과

TCP 접속은 성공했지만 TestClient 화면에서 `InvalidOperationException: You must call the Bind method before performing this operation.` 오류가 발생했다. 서버 쪽 UDP 적용 횟수가 올라가는 것과는 별개로, 클라이언트의 UDP 수신 과정에 문제가 있었다.

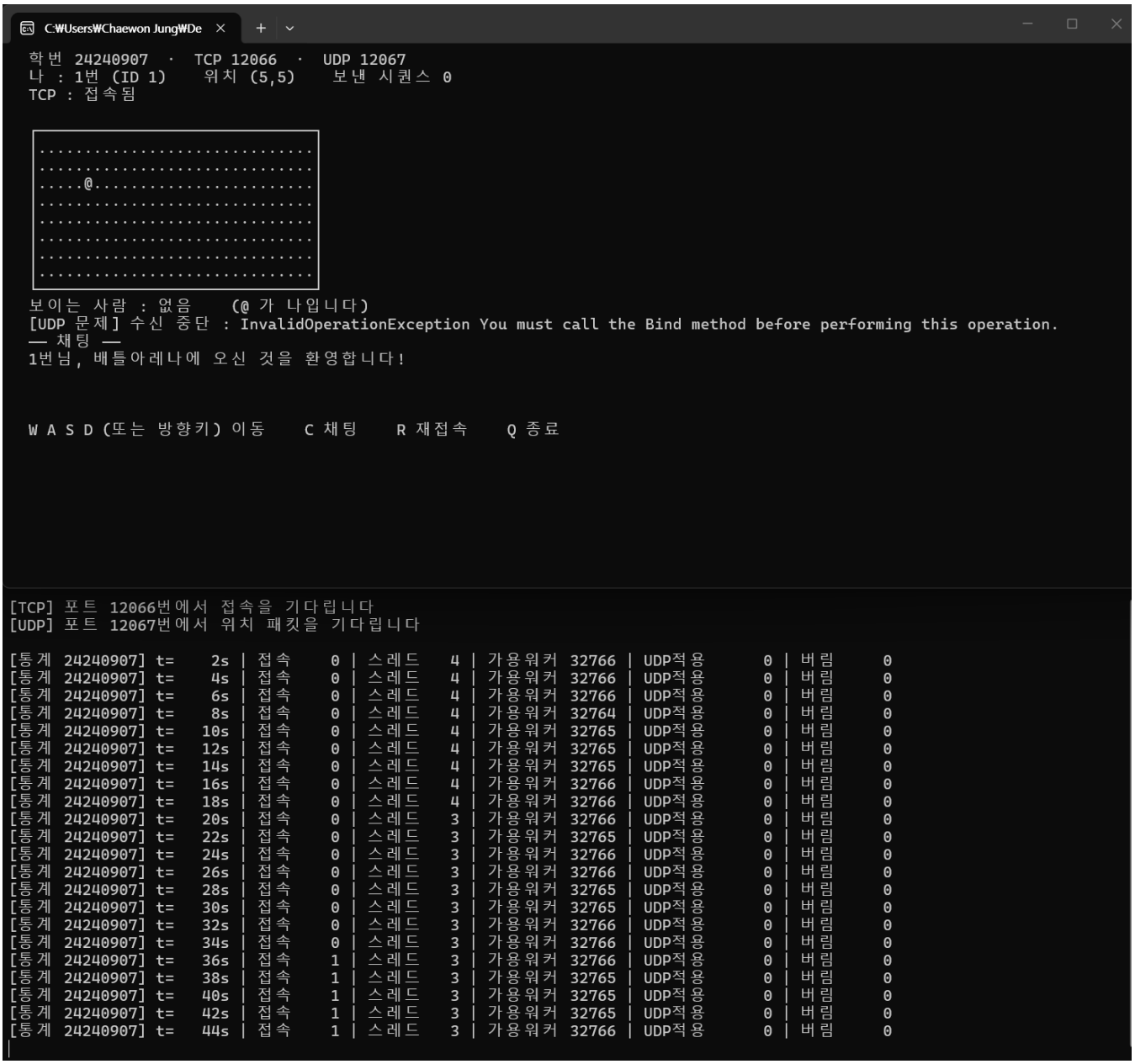


그림 13. A-6 수정 전 — TCP 연결 및 초기 위치 표시 상태

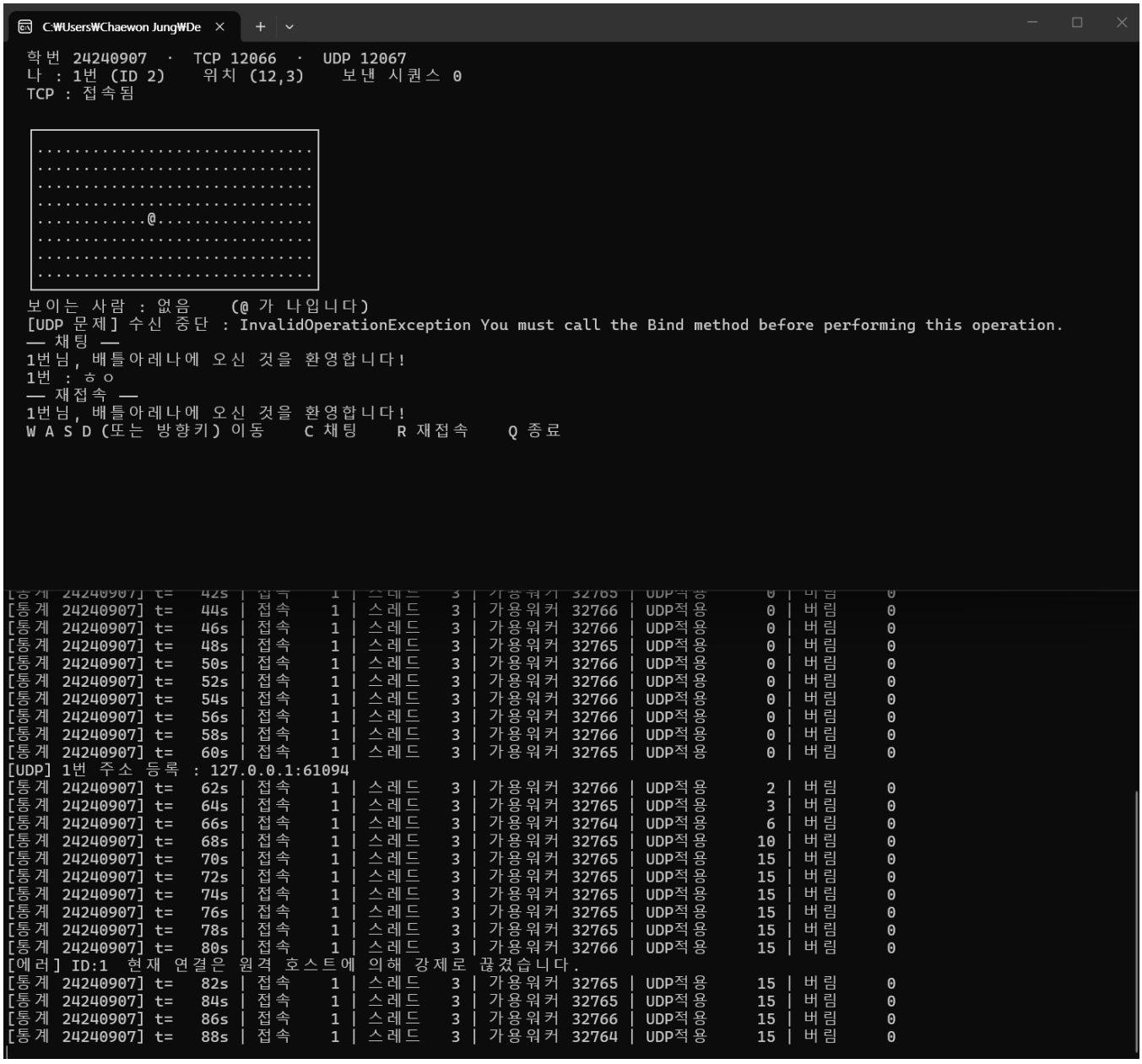


그림 14. A-6 수정 전 — TCP 연결 후 UDP 수신에서 Bind 오류

원인

`TestClient/Program.cs` 원본 153~157 행 부근에서 UDP 소켓은 생성했지만 수신을 시작하기 전에 로컬 주소로 바인딩하지 않았다. 공지문에서 `TestClient` 도 검토 대상으로 지정했으므로 이 부분도 수정 대상에 포함했다.

수정 전 핵심 코드 (`TestClient/Program.cs`, 153~157 행):

```
_udp = new Socket(AddressFamily.InterNetwork,
                  SocketType.Dgram, ProtocolType.Udp);

_udpServer = new IPEndPoint(IPAddress.Parse(SERVER_IP), StudentConfig.UdpPort);
```

수정 후 핵심 코드 (TestClient/Program.cs, 153~159 행):

```
_udp = new Socket(AddressFamily.InterNetwork,
                  SocketType.Dgram, ProtocolType.Udp);

_udp.Bind(new IPEndPoint(IPAddress.Any, 0));

_udpServer = new IPEndPoint(IPAddress.Parse(SERVER_IP), StudentConfig.UdpPort);
```

수정 후 확인

TCP 연결 이후 동일한 조작을 했을 때 이전의 Bind() 오류가 나타나지 않았고, 클라이언트 화면에서 위치 정보를 계속 확인할 수 있었다.

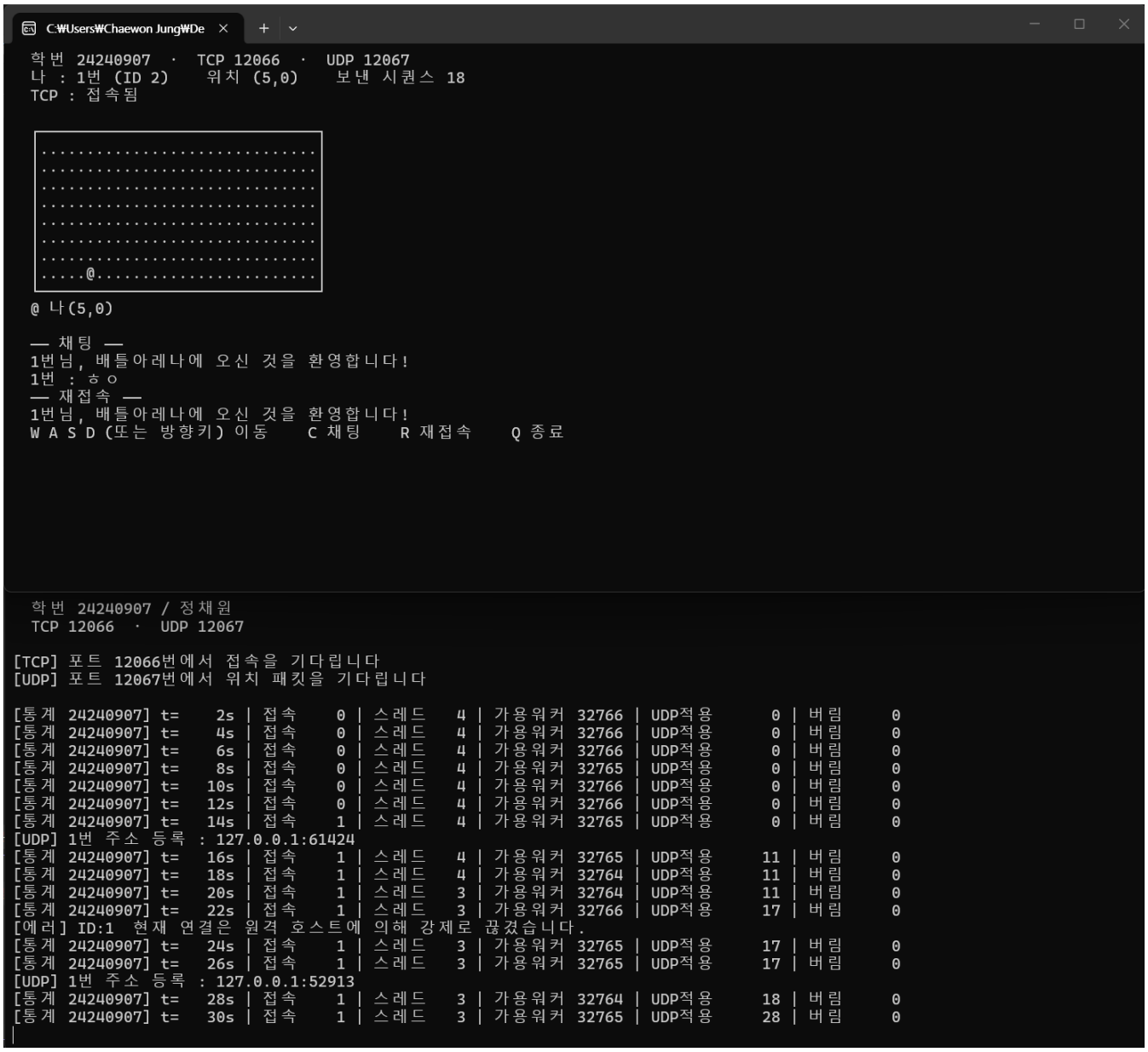


그림 15. A-6 수정 후 — Bind 오류 없이 위치 정보 확인

증거: 02_클라이언트_TCP 연결_UDP 수신오류.png, A_버그 6_수정전_초기상태.png, A_버그 6_수정전_기능테스트.png, A_버그 6_수정후.png

A-7. 사용자가 접속을 종료해도 다른 클라이언트에 퇴장 알림이 나타나지 않는 문제

재현 방법

1. 서버를 실행하고 TestClient 두 개를 BotA, BotB 이름으로 접속시킨다.

2. BotB 를 종료하고 BotA 화면의 채팅·알림 내용을 확인한다.

3. `RemoveSession()`을 수정한 뒤 같은 방법으로 재시험한다.

수정 전 결과

BotA 화면에는 BotB 의 입장 알림은 표시됐지만 BotB 가 종료된 뒤에도 퇴장 알림은 표시되지 않았다. 서버의 접속자 수는 2 명에서 1 명으로 줄었다.

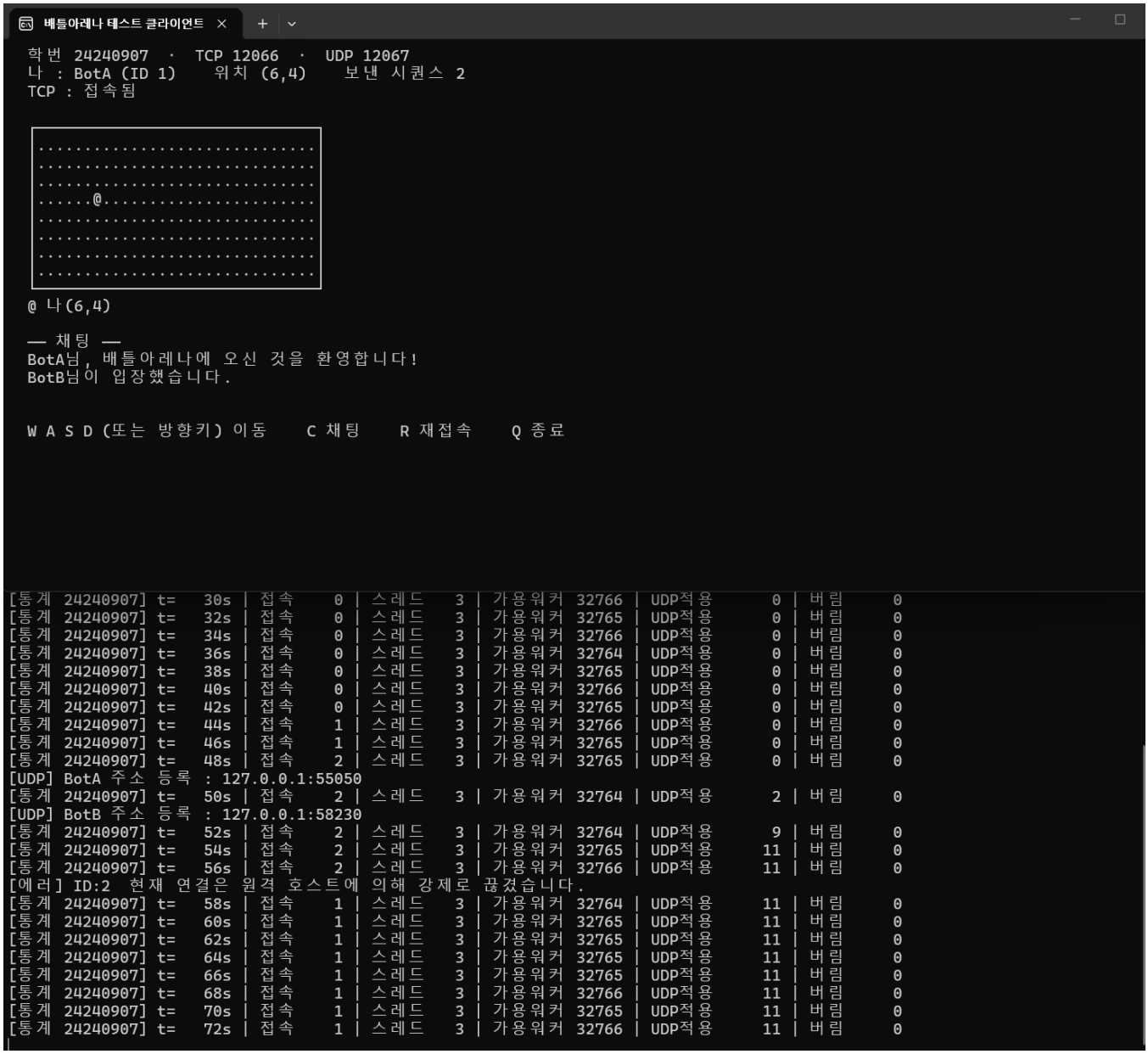


그림 16. A-7 수정 전 — BotB 종료 후 퇴장 알림 누락

원인

OutsourcedServer/GameServer.cs 원본 108 행의 RemoveSession()은 세션을 목록에서 제거하기만 했다. 나머지 접속자에게 PLAYER_OUT 알림을 전송하는 코드가 없었다.

수정 전 (GameServer.cs, 108 행):

```
_sessions.TryRemove(playerId, out _);
```

수정 후 (GameServer.cs, 108~111 행):

```
if (_sessions.TryRemove(playerId, out var session))
{
    Broadcast(new NoticePacket{Type = PacketType.PLAYER_OUT, Message = $"{session.Nickname}님이 퇴장했습니다."});
}
```

수정 후 확인

BotB 를 종료하자 BotA 화면에 BotB 님이 퇴장했습니다.라는 메시지가 표시됐다. 실제로 세션 제거에 성공한 경우에만 퇴장 알림을 보내도록 수정했다.

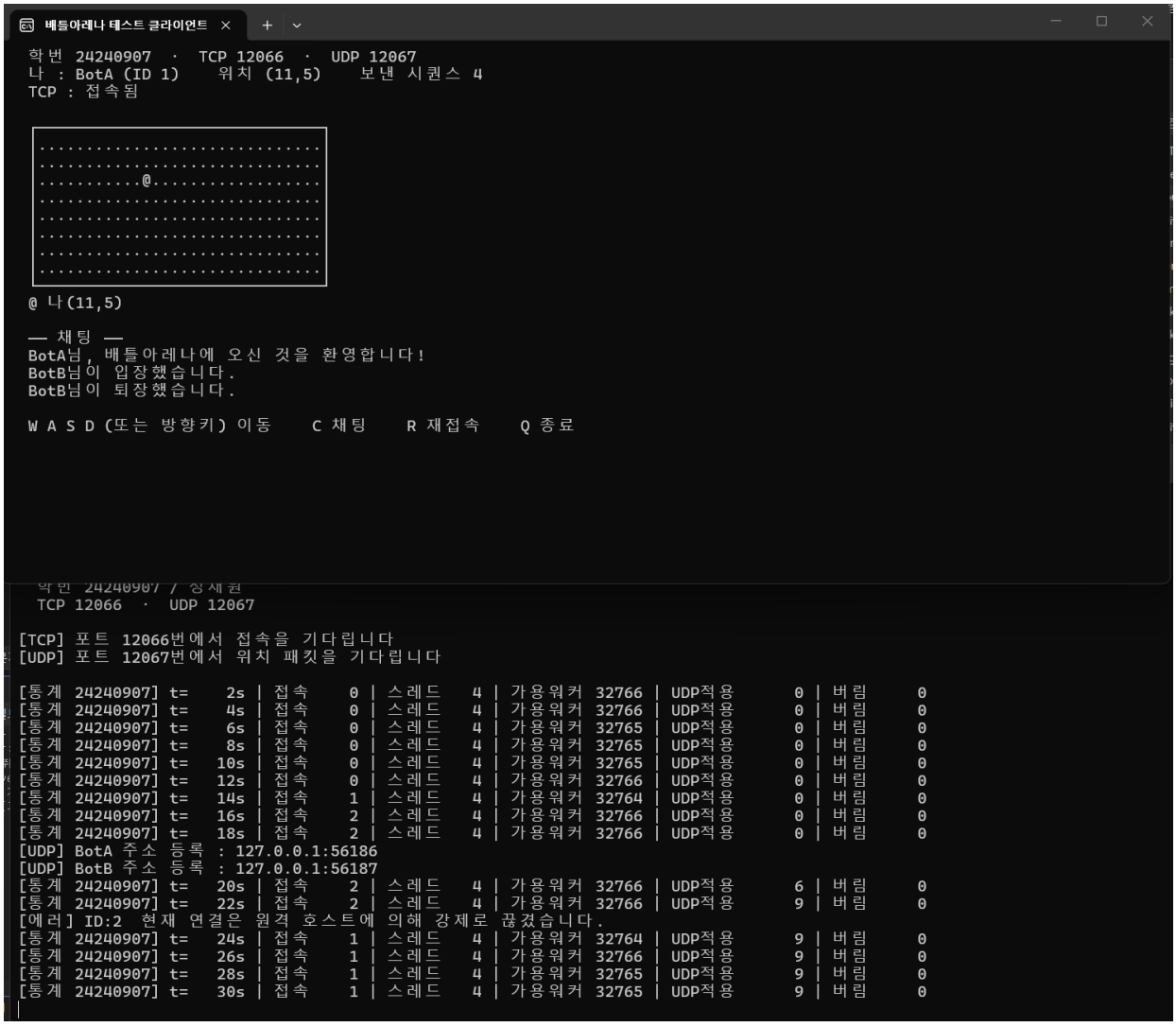


그림 17. A-7 수정 후 — BotB 퇴장 알림 표시

증거: A_버그 7_수정전_퇴장알림누락.png, A_버그 7_수정후_퇴장알림표시.png

Part B. 동시 접속 부하 측정

B-1. 측정 환경과 방법

LoadTester 는 수정하지 않고 납품된 프로그램 그대로 사용했다. 원본과 최종 소스의 `LoadTester/Program.cs`, `LoadTester.csproj` 가 동일한 것도 확인했다. 내 PC 에서 확인한 논리 코어 수는 12 개, .NET 버전은 8.0.32, 운영체제 표시는 `Microsoft Windows NT 10.0.26200.0` 이었다. 테스트에는 내 학번으로 계산된 TCP 12066 번 포트를 사용했다.

1. 수정 전 OutsourcedServer 를 실행한다.

2. LoadTester 에서 PC 정보를 확인하고 1 번 동시 접속 부하를 선택한다.

3. 접속 수를 따로 입력하지 않고 엔터를 눌러 10 명, 50 명, 100 명, 300 명, 500 명을 순서대로 측정한다.

4. 측정 중 서버 콘솔의 `[통계 24240907]`를 확인하고 화면을 저장한다.

5. TCP 수신 방식을 수정한 서버를 실행해 같은 순서로 다시 측정한다.

LoadTester 의 성공은 TCP 접속 후 로그인 패킷을 보내고 10 초 이내에 `WELCOME` 응답을 받은 경우다. CSV 의 평균·최소·최대·p95 시간은 성공한 요청만 대상으로 계산된 값이므로 실패한 요청의 대기 시간까지 포함한 평균은 아니다. 결과는 `측정결과_24240907.csv` 에 기록했다.

CSV 의 스레드 수와 가용워커는 LoadTester 프로세스의 값이며, 서버 콘솔에 표시된 스레드 수와는 구분한다.

증거: `B_수정전_PC 정보.png`, `B_수정전_서버통계.png`, `B_수정후_서버통계.png`, 제출 CSV

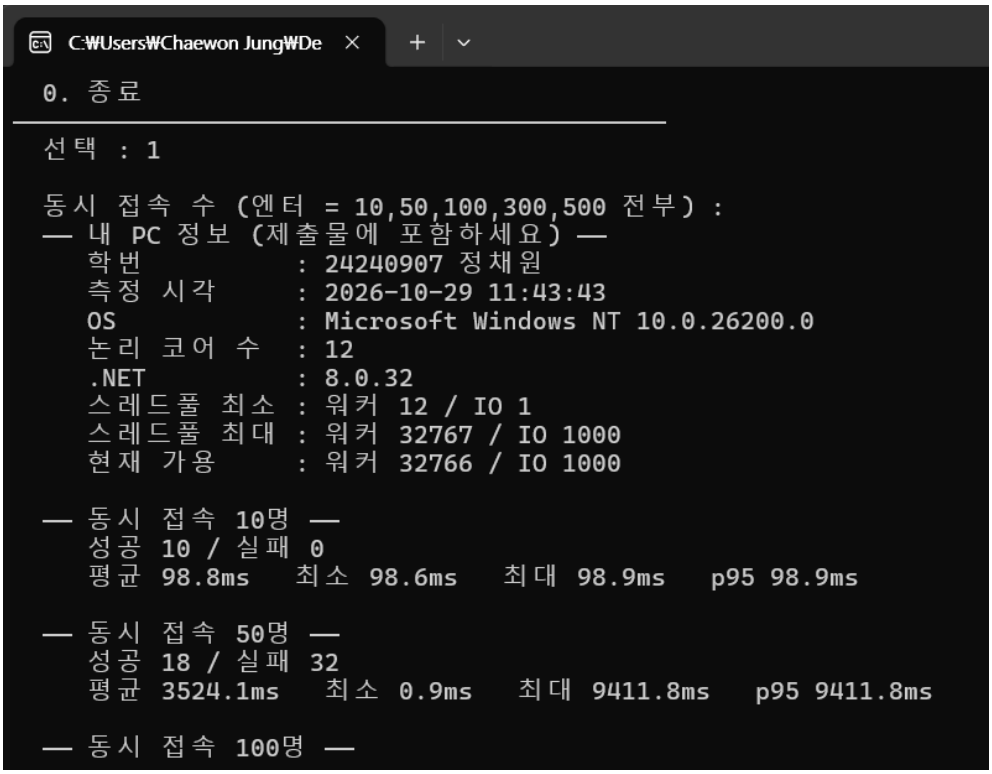


그림 18. B-1 측정 환경 — 논리 코어 12 개와 .NET 8.0.32

B-2. 수정 전·후 측정 결과

표 B-1. 동시 접속 성공·실패 건수

동시 접속	수정 전 성공	수정 전 실패	수정 후 성공	수정 후 실패
10 명	10	0	10	0
50 명	18	32	50	0
100 명	32	68	100	0
300 명	45	255	298	2
500 명	63	437	499	1

표 B-2. 성공한 접속의 응답 시간

동시 접속	수정 전 평균(ms)	수정 전 p95(ms)	수정 후 평균(ms)	수정 후 p95(ms)
10 명	98.8	98.9	151.1	151.1
50 명	3524.1	9411.8	7.8	17.1
100 명	1904.7	8553.5	23.0	58.1
300 명	1173.9	7630.7	215.5	649.1
500 명	1188.8	7281.9	524.6	1214.7

수정 전 측정 시각은 11:43~11:44, 수정 후 제출용 측정 시각은 13:56 이다. 두 결과는 같은 PC 에서 동일한 LoadTester 설정으로 측정했다.

B-3. 성능이 나빠지기 시작한 구간

성능 저하는 동시 접속 50 명 구간부터 뚜렷했다. 10 명에서는 10 명이 모두 성공했지만, 50 명에서는 18 명만 성공하고 32 명이 실패했다. 성공한 요청의 평균 응답 시간도 98.8ms 에서 3524.1ms 로 늘었다. 100 명에서는 32 명, 300 명에서는 45 명, 500 명에서는 63 명만 성공해 접속 수를 늘려도 성공 건수가 비례해서 증가하지 않았다.

특히 500 명 구간은 성공 63 명, 실패 437 명으로 성공률이 12.6%였다. 이 결과만으로 원본 서버를 안정적인 동시 접속 서버라고 판단하기는 어렵다.

B-4. 성능 저하 원인

가장 큰 원인은 OutsourcedServer/PacketHelper.cs 원본의 TCP 수신 방식으로 판단했다. 원본 51 행과 63 행에서는 블로킹 함수인 socket.Receive()를 Task.Run()으로 감싸고 있었다.

```
int n = await Task.Run(() => socket.Receive(header, 0, 4, SocketFlags.None));
await Task.Run(() => socket.Receive(body, 0, bodyLength, SocketFlags.None));
```

Task.Run 을 붙였다고 소켓 수신 자체가 비동기로 바뀌는 것은 아니다. Receive()는 데이터가 도착할 때까지 해당 작업을 실행하는 스레드를 기다리게 한다. 접속자마다 TCP 패킷을 받는 작업이 생기므로 클라이언트가 많아지면 스레드 사용량과 대기 작업이 늘고, 새 접속의 로그인 응답 처리까지 늦어질 수 있다.

실제 수정 전 서버 통계 화면에는 스레드 수가 58 개, 83 개, 86 개, 92 개로 증가한 구간이 기록되어 있다. 다만 이 화면의 가용워커가 0 이 된 것은 확인되지 않았으므로, 스레드풀이 완전히 고갈됐다고 단정하지는 않았다. 코드 구조와 접속 성공률·응답 지연을 함께 보았을 때, 블로킹 수신이 큰 부담을 준다고 판단했다.

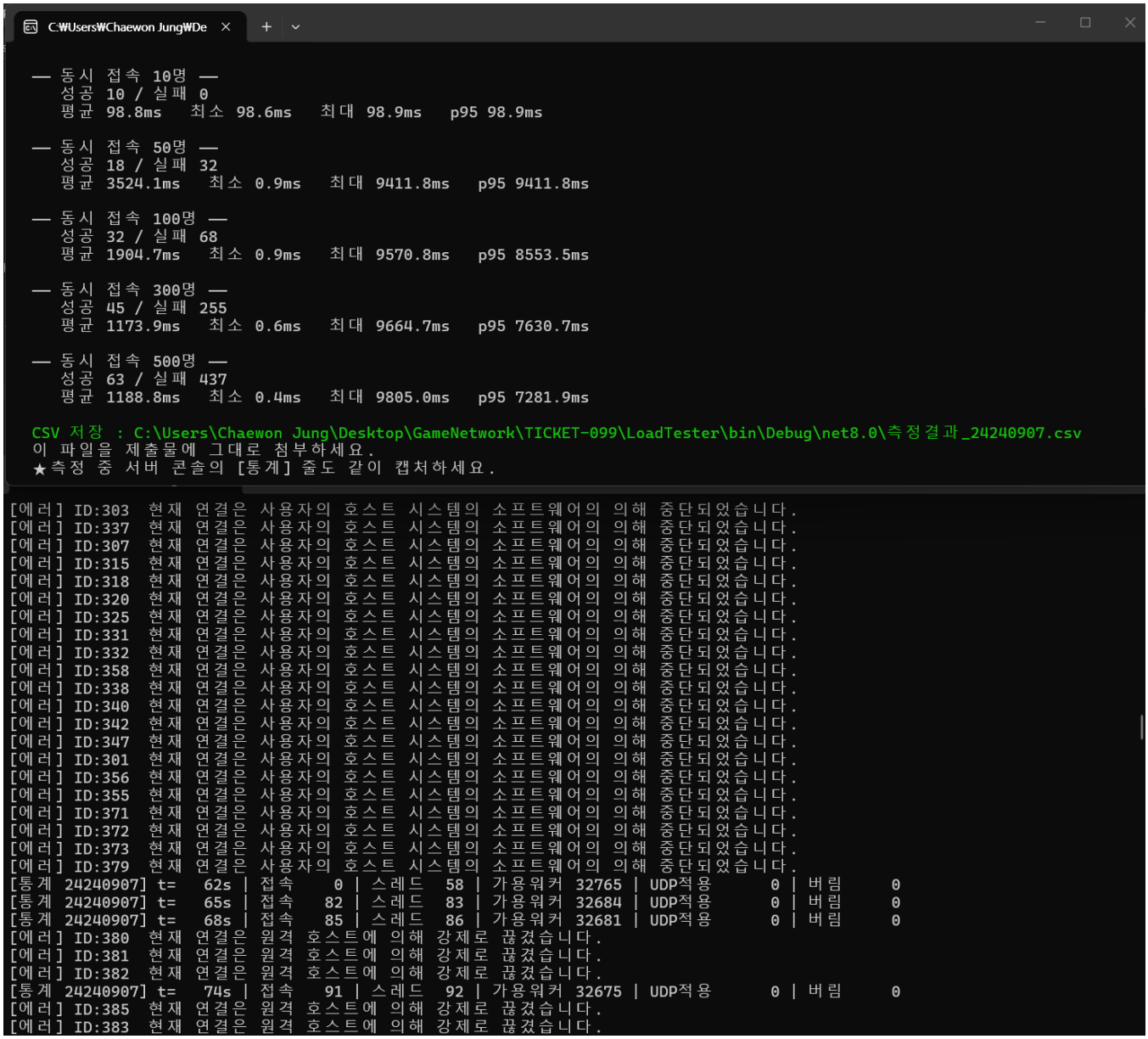


그림 19. B-4 수정 전 부하 — 접속 결과와 서버 스레드 증가

B-5. 수정 내용과 재측정 결과

`PacketHelper.cs` 에서 `Task.Run(() => socket.Receive(...))`를 없애고 `socket.ReceiveAsync()`를 직접 기다리도록 수정했다. 수신한 바이트 수를 누적해 헤더와 본문을 필요한 길이만큼 끝까지 읽는 `ReadAsync()` 함수도 추가했다. 이 수정은 Part A 의 TCP 분할 수신 문제와 부하 문제를 함께 다룬다.

수정 후 50 명과 100 명 구간에서는 모두 접속에 성공했고, 300 명은 298 명, 500 명은 499 명이 성공했다. 500 명 구간 성공률은 99.8%였다. 원본에서 437 명이 실패하던 구간이 수정 후에는 1 명 실패로 줄어든 점이 가장 큰 차이였다.

수정 후 별도로 진행한 추가 측정 화면에서는 500 명 중 499 명이 성공했고, 서버가 부하를 처리하는 도중 [통계]에 현재 접속 432 명, 스레드 12 개가 표시됐다. 이 화면은 제출 CSV 의 13:56 실행이 아니라 별도 실행의 보조 증거다. 따라서 추가 화면의 300 명 결과(299 명 성공)를 제출 CSV 의 300 명 결과(298 명 성공)와 한 번의 실행처럼 섞지 않았다.

모든 항목이 좋아졌다고 볼 수는 없다. 10 명 구간의 평균 응답 시간은 수정 전 98.8ms, 수정 후 151.1ms 로 오히려 늘었다. 측정이 한 차례씩인 만큼 그 차이의 원인을 단정하기는 어렵다. 다만 50 명 이상에서의 접속 성공률과 성공한 요청의 응답 시간은 확실히 개선된 결과를 확인했다. 500 명에서도 실패 1 건이 있었으므로 완전히 실패가 없는 서버라고 결론내리지는 않았다.

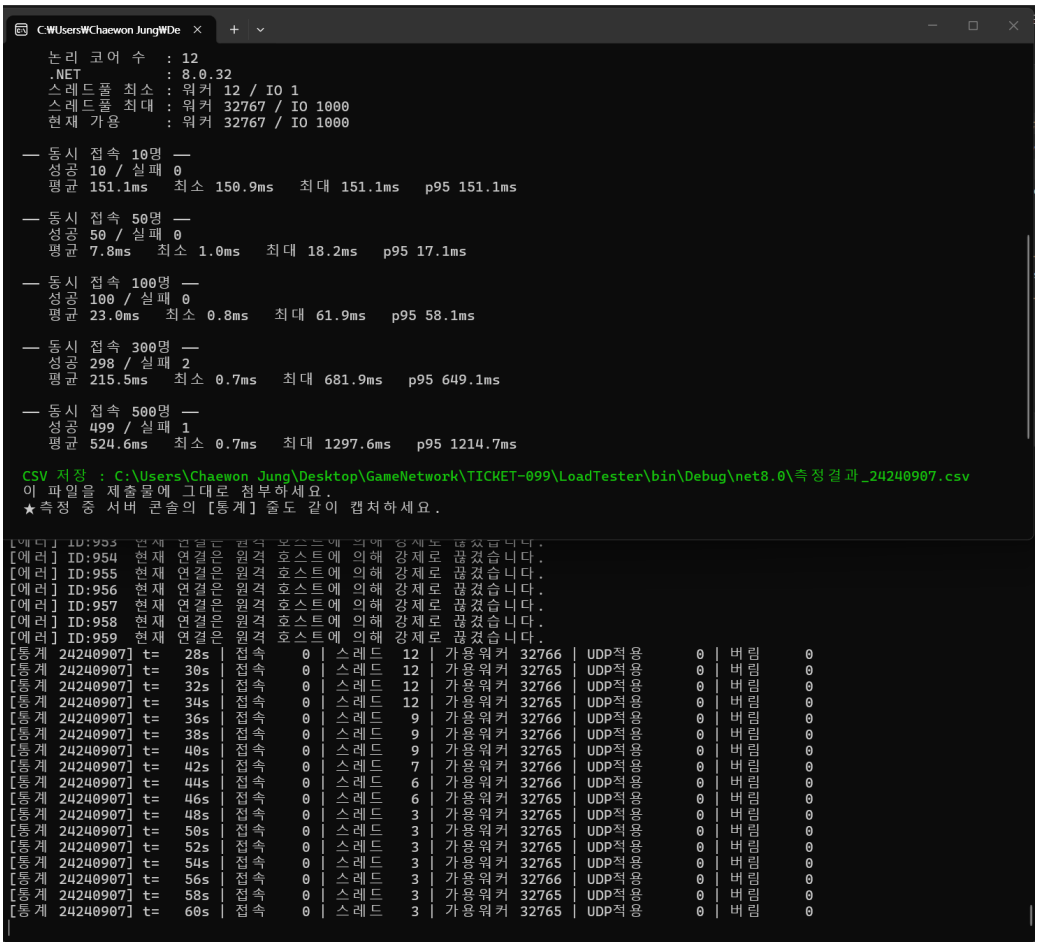


그림 20. B-5 수정 후 제출용 측정 — CSV 의 13:56 실행 결과

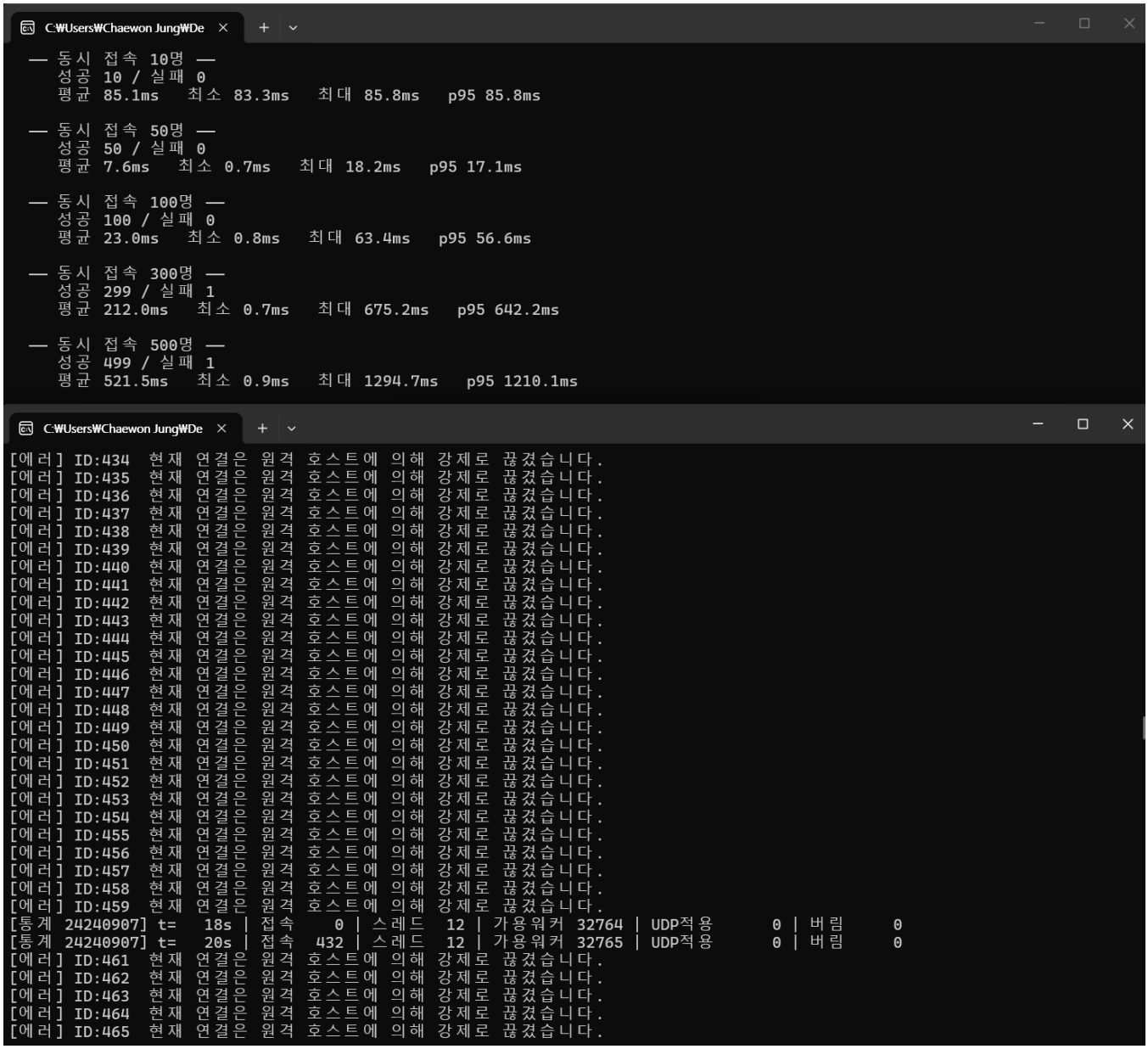


그림 21. B-5 별도 추가 실행 — 현재 접속 432 명, 서버 스레드 12 개

증거: B_수정전_서버통계.png, B_수정후_서버통계.png, B_수정후_측정중통계.png, 측정결과_24240907.csv

Part C. 업체 검토서 판정

C-1. VendorReview.docx 의견 10 건 판정

번호	업체 의견	판정	근거
1	TCP 와 UDP 를 나눈 구조가 적절하다	동의	로그인·채팅은 신뢰성이 중요하고 위치는 최신 값이 중요하므로 이 프로젝트에 맞는 역할 분리다.
2	STATE 패킷을 한 번에 묶어 보낸다	동의	전체 위치를 한 번에 묶으면 접속자마다 별도 패킷을 만드는 방식보다 전송 횟수를 줄일 수 있다.
3	`session.RunAsync()`에 `await`가 빠졌다	반대	접속을 받는 반복문에서 이 작업의 종료를 기다리면 현재 클라이언트가 나갈 때까지 다음 접속 처리가 지연된다.
4	빈 `catch` 에 모든 UDP 실패 로그와 재시도를 넣어야 한다	반대	UDP 위치 정보는 약 33ms 마다 새로 전송되므로, 실패할 때마다 재시도하거나 로그를 출력하기보다 반복되는 오류를 모아 기록하는 편이 적절하다.
5	`Task.Delay(33)`은 정밀한 30fps 타이머가 아니다	동의	33ms 는 33.33ms 와 다르고 `Task.Delay` 도 실행 지연의 영향을 받으므로 정확히 30fps 를 보장하지 않는다.
6	`MAX_BODY` 를 8192 에서 65535 로 올려야 한다	반대	8192 는 패킷 크기를 제한하는 정책이기도 하다. 긴 채팅을 받기 위해 무조건 상한을 키우면 큰 패킷에 필요한 메모리도 증가한다.
7	`Listen(512)`는 과도하게 메모리를 예약한다	반대	`Listen` 의 인수는 연결 대기열 크기와 관련된 값이며, 512 명분 세션을 미리 생성한다는 뜻은 아니다.
8	`NoDelay = true` 는 대역폭 때문에 꺼야 한다	반대	Nagle 알고리즘을 끄면 작은 패킷이 늘 수 있지만, 로그인·채팅 등의 지연을 줄이는 목적도 있으므로 무조건 나쁜 설정은 아니다.
9	`StringBuilder` 를 필드에 둔 것 자체가 스레드 안전성 오류다	반대	이 코드의 `_stateBuilder` 는 하나의 `BroadcastLoopAsync()`에서 사용된다. 필드라는 이유만으로 동시 접근이 발생한다고 볼 수 없다.
10	비동기 구조이므로 부하 테스트는 필요 없다	반대	원본에는 블로킹 `Receive()`가 있었고, 실제 50 명 부하부터 접속 실패가 크게 늘었다.

업체는 치명적 결함이 없고 인수가 가능하다고 결론 내렸지만, 실제 실행에서는 서버 종료와 패킷 처리 오류, 다수 접속 실패가 확인됐다. 따라서 원본을 그대로 인수해도 된다는 결론에는 동의하기 어렵다.

C-2. 반대 의견 실행 검증 ① — 의견 3 번의 await 추가

업체 주장

`GameServer.StartAsync()`에서 `_ = session.RunAsync();`로 호출하는 대신 `await session.RunAsync();`로 바꿔야 한다.

실험 방법

1. 원래 코드로 서버를 실행해 BotA 와 BotB 를 함께 접속시킨다.
2. 두 클라이언트가 정상적으로 ID 를 받고 입장하는지 확인한다.
3. `GameServer.cs` 의 해당 호출을 업체 권고대로 임시로 `await session.RunAsync();`로 바꾼 뒤 다시 두 명을 접속시킨다.

실제 결과

원래 코드에서는 BotA 와 BotB 가 각각 ID 를 받고 접속했으며 서버 통계에서도 접속자 2 명이 확인됐다. 반면 `await` 를 적용한 실험에서는 BotA 의 처리가 끝나기를 기다리면서 뒤에 접속한 BotB 의 로그인 처리가 진행되지 않았고, BotB 화면에 ID 0 이 표시됐다. 서버 접속자 수도 1 명으로 나타났다.

판단

이 부분은 세션 작업이 끝날 때까지 기다리는 것이 아니라 여러 세션을 동시에 진행시키도록 설계된 곳이다. 따라서

`await` 가 없다는 사실만으로 오류라고 할 수 없다. 해당 실험 변경은 원상복구했으며, 최종 수정 코드에는 업체의 `await` 권고를 적용하지 않았다.

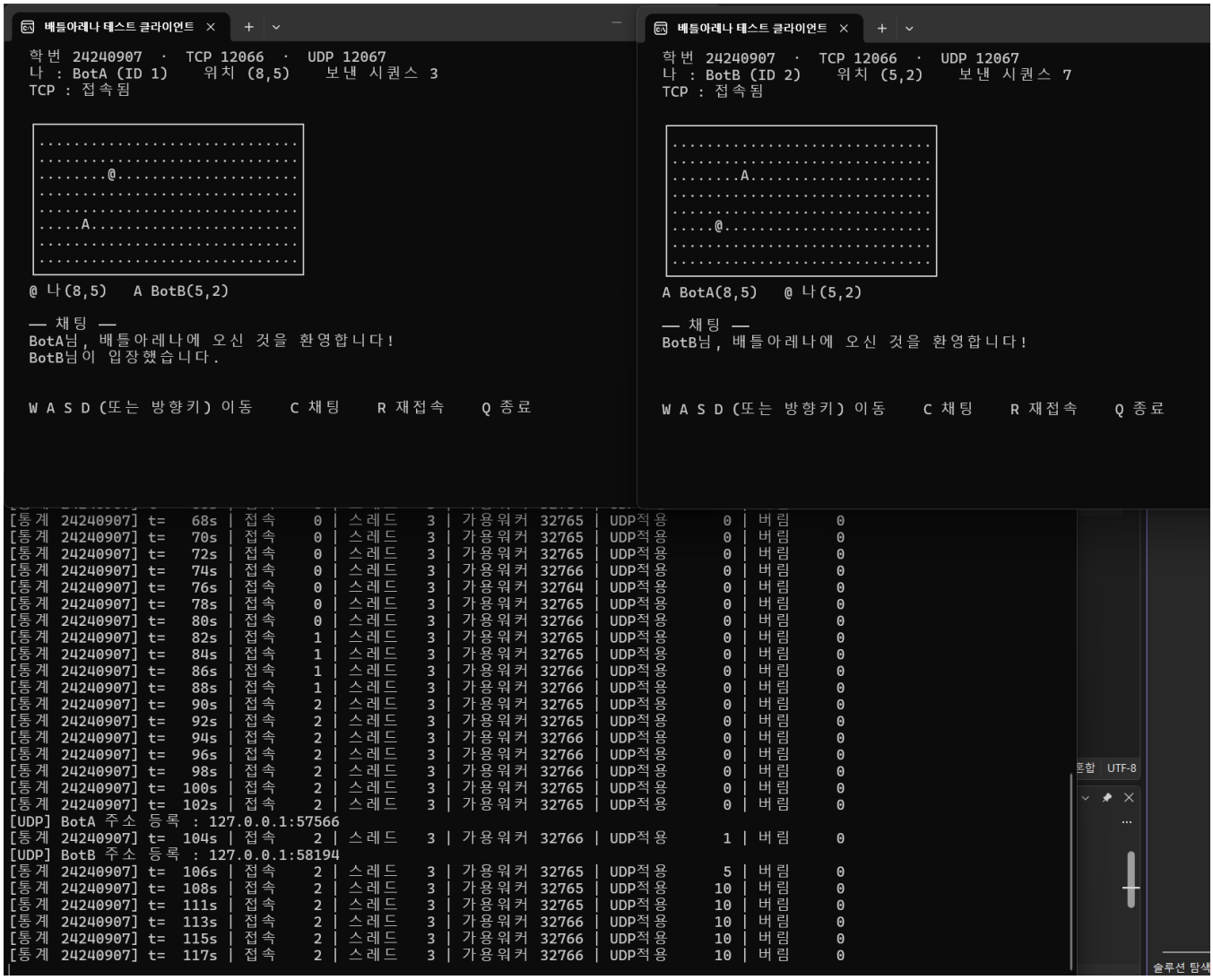


그림 22. C-2 기존 코드 — BotA·BotB ID 부여 및 서버 접속자 2 명

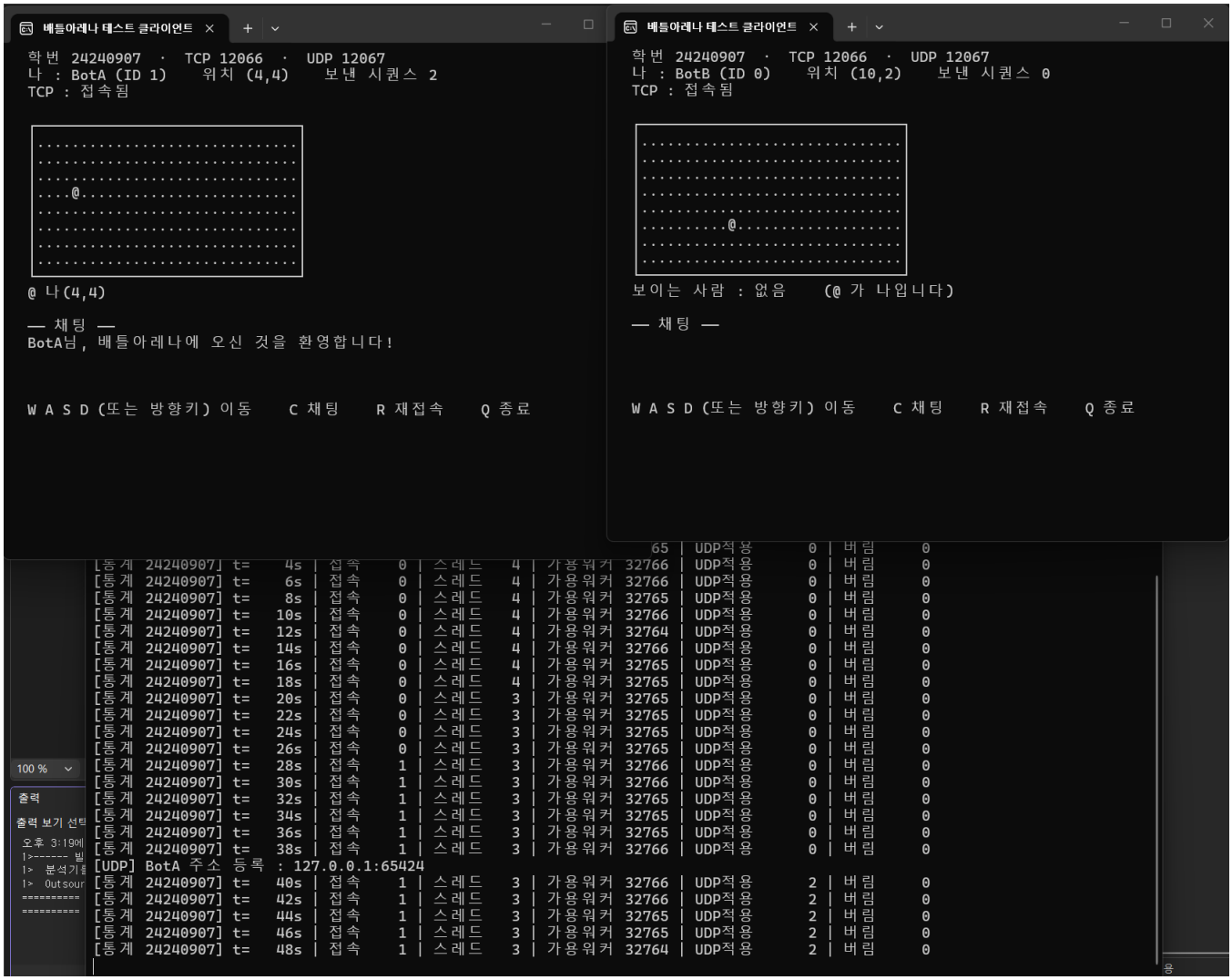


그림 23. C-2 await 임시 적용 — BotB ID 0, 서버 접속자 1 명

증거: C_의견 3_기존코드_동시접속정상.png, C_의견 3_await 적용_동시접속검증.png

C-3. 반대 의견 실행 검증 ② — 의견 10 번의 동시 접속 안정성 주장

업체 주장

수신 처리를 비동기로 구현했으므로 동시 접속자가 늘어나도 안정적으로 처리할 수 있고 별도의 부하 테스트가 필요하지 않다.

실험 방법

1. 수정 전 서버를 실행한다.

2. LoadTester 1 번 메뉴로 동시 접속 10 명, 50 명, 100 명, 300 명, 500 명을 순서대로 시험한다.

3. 접속 성공·실패 건수와 서버 스레드 통계를 기록한다.

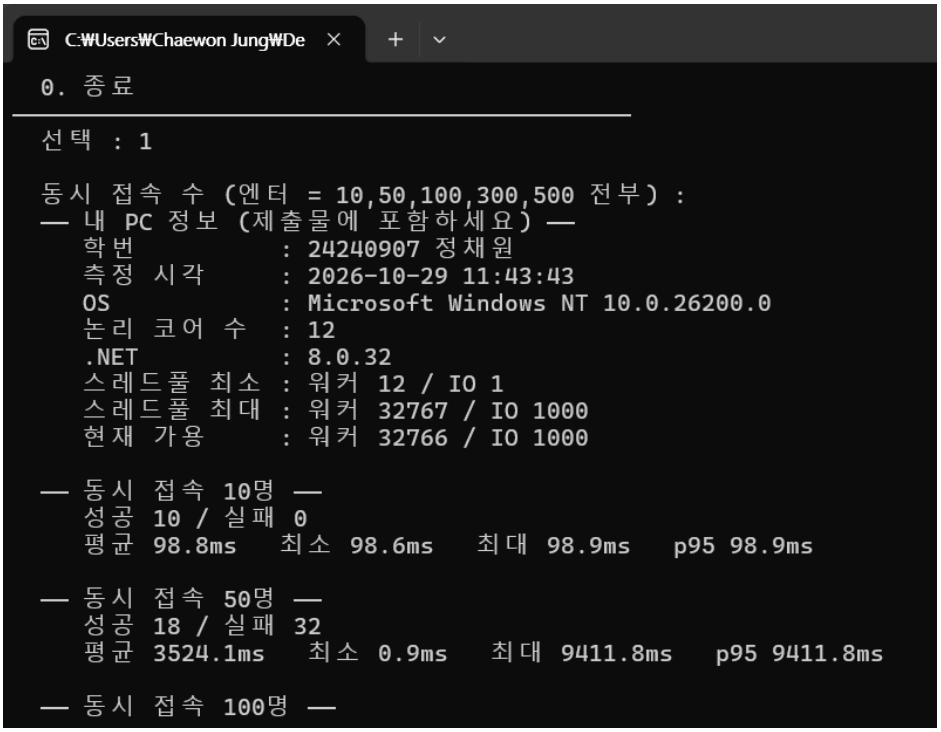
실제 결과

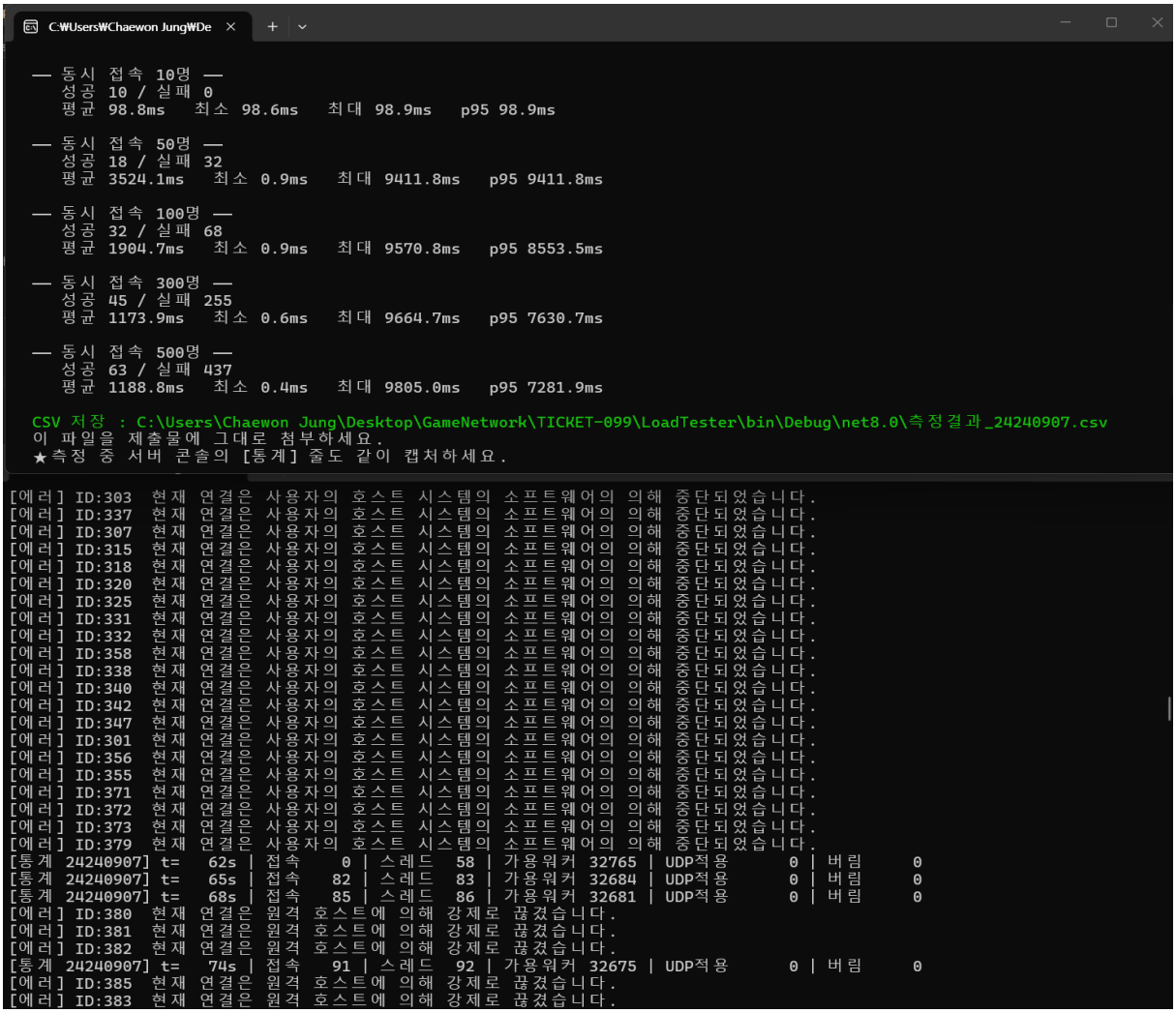
10 명에서는 10 명 모두 성공했으나 50 명에서는 18 명만 성공했다. 100 명은 32 명, 300 명은 45 명, 500 명은 63 명만 성공했다. 수정 전 서버 콘솔에서는 스레드 수가 92 개까지 증가한 화면도 확인됐다.

판단

코드에 비동기 메서드가 있어도 내부에서 블로킹 수신 함수를 사용하면 다수 접속에서 병목이 생길 수 있다. 따라서 업체의 동시 접속 안정성 주장은 실제 측정 결과와 맞지 않는다. 이 실험은 Part B 의 수정 전 부하 측정 자료를 함께 사용했다.

증거: B_수정전_PC 정보.png, B_수정전_서버통계.png, 측정결과_24240907.csv





동시 접속 성공·실패 건수

동시 접속	수정 전 성공	수정 전 실패	수정 후 성공	수정 후 실패
10 명	10	0	10	0
50 명	18	32	50	0
100 명	32	68	100	0
300 명	45	255	298	2
500 명	63	437	499	1

성공한 접속의 응답 시간

동시 접속	수정 전 평균(ms)	수정 전 p95(ms)	수정 후 평균(ms)	수정 후 p95(ms)
10 명	98.8	98.9	151.1	151.1
50 명	3524.1	9411.8	7.8	17.1
100 명	1904.7	8553.5	23.0	58.1
300 명	1173.9	7630.7	215.5	649.1
500 명	1188.8	7281.9	524.6	1214.7

Part D. 작업 과정 기록

D-1. Git 작업 기록

과제 작업은 기존 GameNetwork 저장소의 `ticket-099-review` 브랜치에서 진행했다. 원본을 먼저 추가한 뒤 학번과 이름을 설정하고, 버그별로 수정 내용을 나눠 커밋했다. 과제 관련 커밋은 총 9 개이며, 각 커밋 메시지에 변경 내용을 적었다.

저장소 주소: https://github.com/projcw/GameNetwork

작업 브랜치: `ticket-099-review`

최종 커밋: `cb79d69`

순서	커밋	주요 내용
1	`7f79d1d`	TICKET-099 외주 납품 원본 추가
2	`f4a8ff9`	학번·이름 설정 및 빌드 산출물 추적 제외
3	`29d2e2e`	TCP 분할 수신 시 전체 바이트 읽기
4	`3aa029a`	비정상 패킷 처리 시 서버 종료 방지
5	`bf73e33`	UDP 중복 패킷 처리 수정
6	`960fcc2`	UDP 수신 데이터 길이 처리 수정
7	`3d7cc1a`	UDP 사용자 주소 불일치 패킷 차단
8	`901b9d5`	UDP 수신 소켓 바인딩 누락 수정
9	`cb79d69`	접속 종료 시 퇴장 알림 누락 수정

커밋 내역 전체는 `git.txt` 에 저장했고, `git log --oneline --graph` 출력도 캡처했다.

```
* cb79d69 (HEAD -> ticket-099-review, origin/ticket-099-review) 접속 종료 시 퇴장 알림 누락 수정
* 901b9d5 UDP 수신 소켓 바인딩 누락 수정
* 3d7cc1a UDP 사용자 주소 불일치 패킷 차단
* 960fcc2 UDP 수신 데이터 길이 처리 수정
* bf73e33 UDP 중복 패킷 처리 수정
* 3aa029a 비정상 패킷 처리 시 서버 종료 방지
* 29d2e2e TCP 분할 수신 시 전체 바이트를 읽도록 수정
* f4a8ff9 TICKET-099 학번, 이름 설정 및 빌드 산출물 추적 제외
* 7f79d1d TICKET-099 외주 납품 원본 추가
*   60b3716 (origin/main, origin/HEAD, main) Merge branch 'main' of https://github.com/projcw/GameNetwork
|\
| * a1e53c4 Delete .vs directory
| * 1402048 패킷 추가
| * 0d55944 Delete KakaoTalk_20260918_095704387.png
* | 705ef1d add: 4주차 실습
|/
*   bc33a9b Merge branch 'main' of https://github.com/projcw/GameNetwork
|\
| * 8e52674 패킷 5가지
| * 54cb3fe Rename 서버 기능 정의 명세서.png to 서버 기능 정의_명세서.png
| * 628fb72 Rename KakaoTalk_20260911_115227676.png to 서버 기능 정의 명세서.png
| * b341b58 Add files via upload
* | 8c08415 핵심 패킷
|\|
| *   f61062b 수정
| |\
* | \   7be5c37 서버 기능 정의 명세서 수정
|\ \ \
| |/ /
|/| /
| |/
| *   314aaa6 서버 명세서
| |\
* | \   4c4a106 서버 명세서
|\ \ \
| |/ /
|/| /
| |/
| * fc784c8 Update README.md
| * cfcd21d Add 정채원 to team members list
* | 9f41d66 명세서
|/
* f9cd6dd 팀원 수정
* 4f8fe7b 팀원 수정
* b75663a Initial commit
```

그림 24. D-1 ticket-099-review 브랜치의 과제 관련 커밋 9 개

증거: `D_최종_커밋기록.png`, `git.txt`

D-2. 막혔던 일 ① — TCP 는 연결됐지만 UDP 수신이 되지 않았던 문제

처음 생각

TestClient 한 명을 접속시키면 TCP 연결과 위치 정보 수신까지 정상적으로 확인할 수 있을 것으로 예상했다.

확인 방법

서버를 실행하고 TestClient 한 명을 접속시켜 화면을 확인했다. TCP 는 연결됐지만 UDP 수신 과정에서 오류가 발생했다. `TestClient/Program.cs` 에서 UDP 소켓을 생성하고 수신하는 부분을 확인하고, 5 주차 수업의 UDP 소켓 초기화 방법도 확인할 필요가 있다고 생각했다.

실제 결과

클라이언트 화면에 `You must call the Bind method before performing this operation.` 오류가 표시됐다. TCP 연결은 성공했기 때문에 서버 전체가 접속되지 않는 문제는 아니었다. 코드를 확인하니 UDP 수신 소켓을 만든 뒤 `Bind()`를 호출하는 부분이 빠져 있었다.

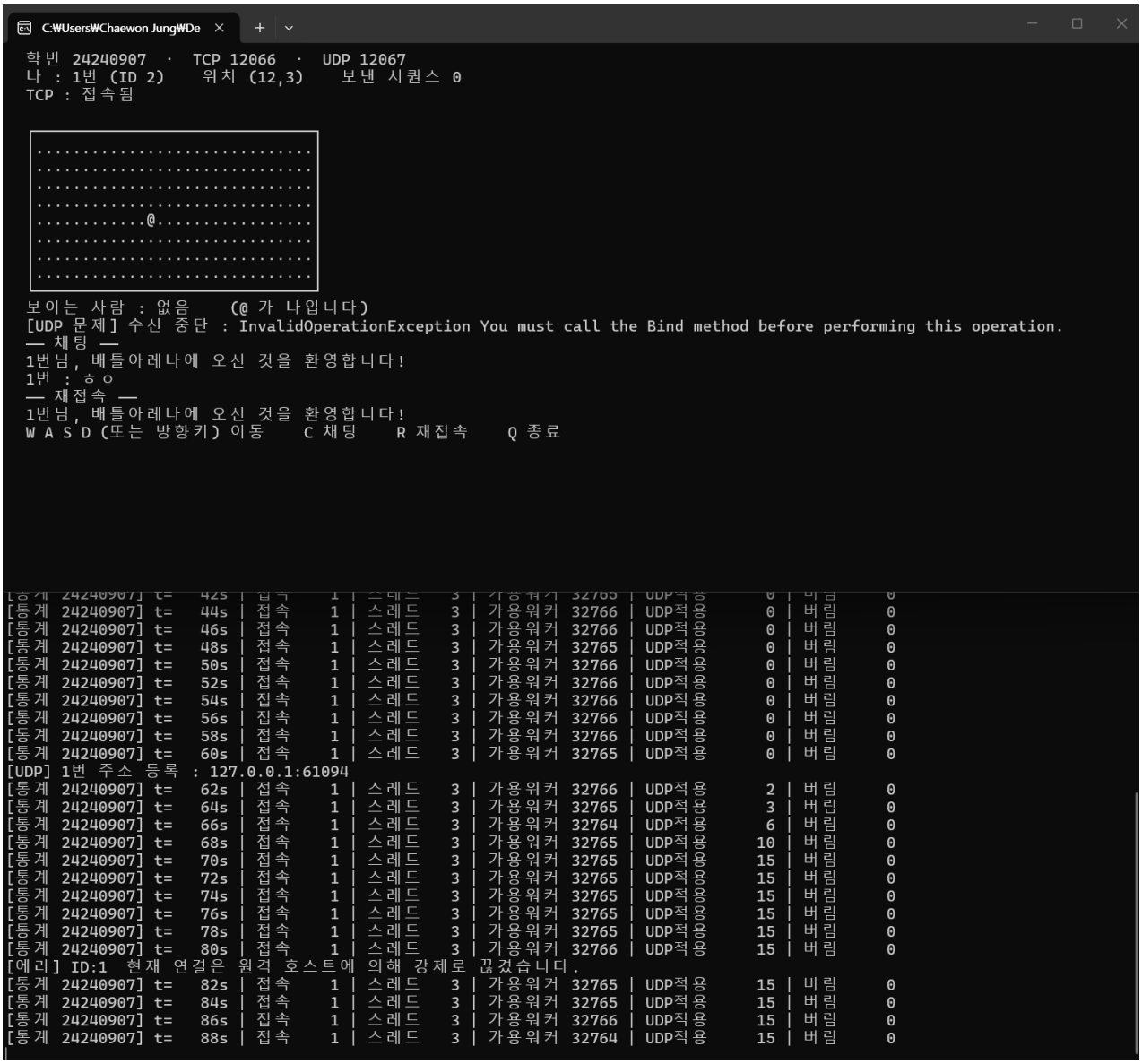


그림 25. D-2 수정 전 — TCP 연결 이후 UDP Bind 오류

다음 생각과 해결

UDP 는 TCP 접속과 별도로 수신할 로컬 주소를 준비해야 하는지 확인할 필요가 있다고 판단했다. UDP 소켓에 `Bind(new IPEndPoint(IPAddress.Any, 0))`를 추가한 뒤 다시 테스트했고, 이전 오류가 나타나지 않는 것을 확인했다. 처음에는 TCP 접속 성공만 확인했지만, 이후에는 TCP 연결 상태와 UDP 수신 상태를 나눠서 살펴보게 됐다.

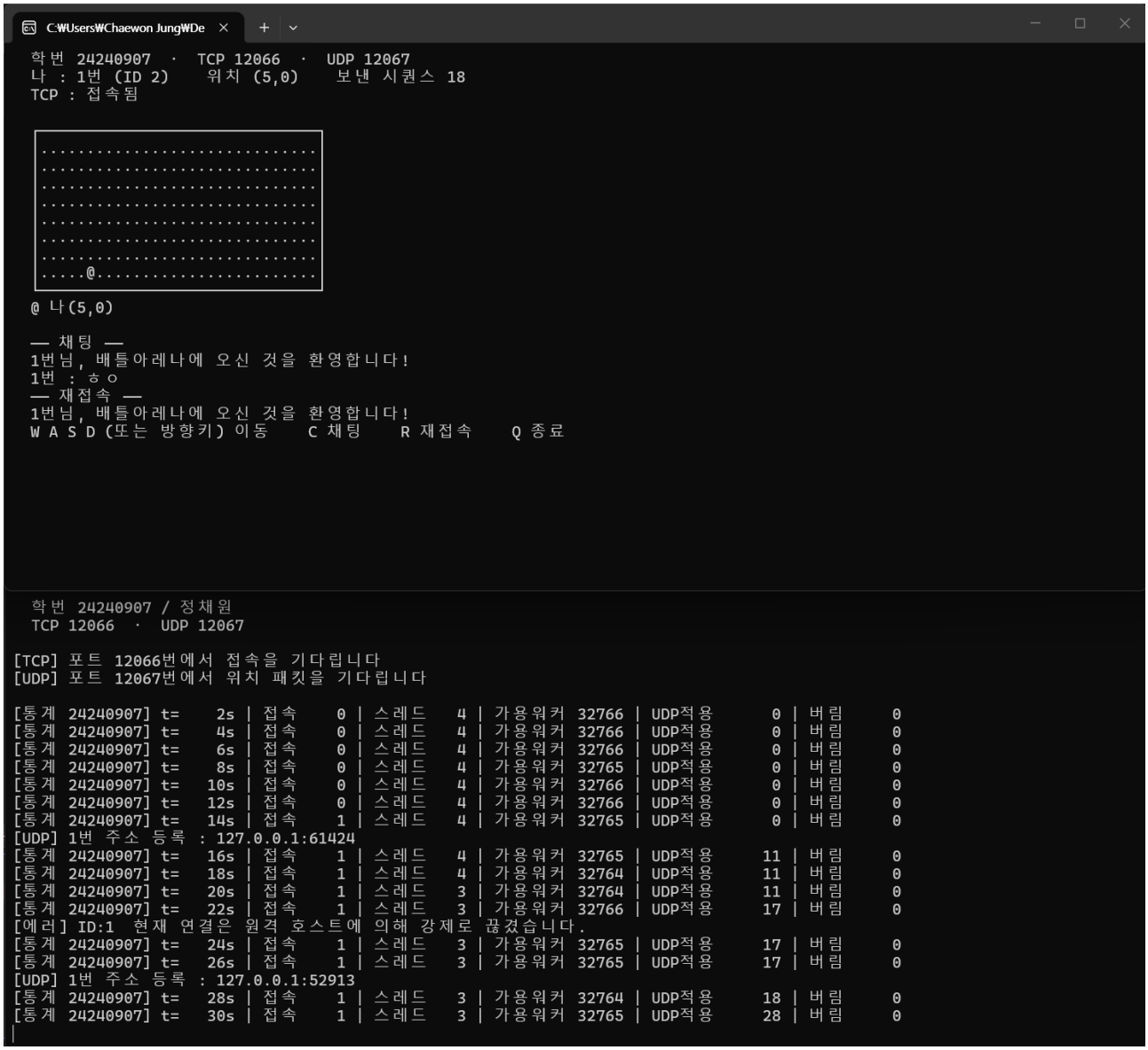


그림 26. D-2 수정 후 — Bind 수정 후 UDP 수신 오류 해결

증거: `02_클라이언트_TCP 연결_UDP 수신오류.png`, `A_버그 6_수정전_기능테스트.png`, `A_버그 6_수정후.png`

D-3. 막혔던 일 ② — 동시 접속 수를 늘리자 실패가 급증한 문제

처음 생각

업체 검토서에는 비동기 방식이라 동시 접속에 문제가 없다고 적혀 있었다. 처음 10 명을 접속시켰을 때도 모두 성공해서 더 많은 접속도 비슷하게 처리될 것으로 예상했다.

확인 방법

LoadTester 로 접속 수를 10 명에서 50 명, 100 명, 300 명, 500 명까지 늘리며 성공·실패 건수를 기록했다. 동시에 서버 통계의 스레드 수를 확인하고, 접속 처리 중 어디서 작업이 기다리는지 코드를 살펴봤다.

실제 결과

10 명은 모두 성공했지만 50 명에서 성공 18 명, 실패 32 명이 나왔다. 500 명에서는 성공 63 명, 실패 437 명이었고 서버 화면에서 스레드 수가 크게 늘어나는 것도 보였다. `PacketHelper.cs` 를 확인하니 비동기 함수 안에서 블로킹 `Receive()`를 `Task.Run()`으로 실행하고 있었다.

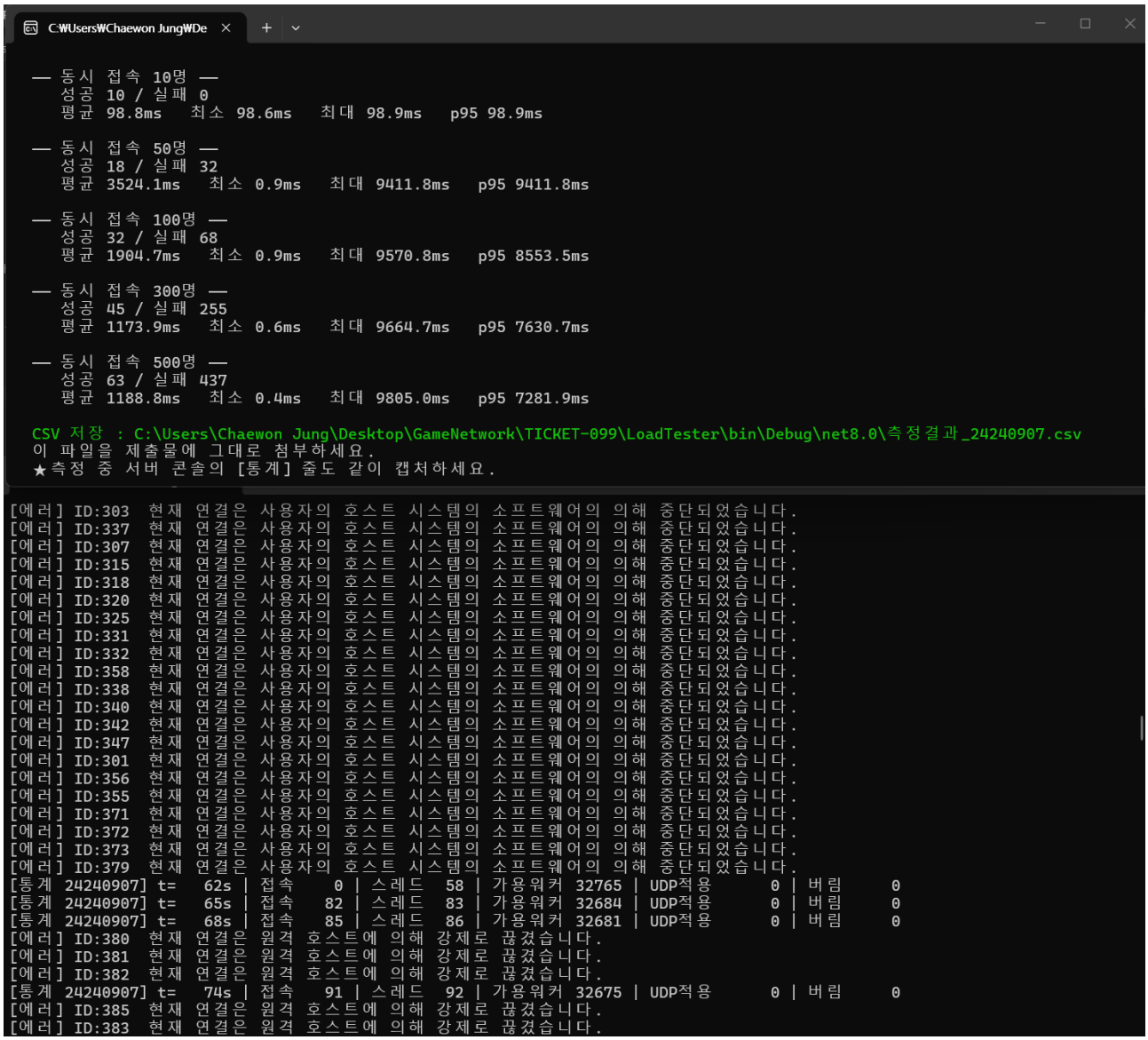


그림 27. D-3 수정 전 — 동시 접속 실패와 서버 스레드 증가

다음 생각과 해결

비동기 함수를 썼다고 해서 그 안의 모든 작업이 스레드를 점유하지 않는 것은 아니라는 점을 알게 됐다. `ReceiveAsync()`를 사용하고 필요한 바이트를 모두 읽을 때까지 반복하도록 수정했다. 다시 측정하니 50 명은 모두 성공했고, 500 명 중에는 499 명이 성공했다. 이후에는 코드를 볼 때 메서드 이름보다 실제로 어떤 함수가 데이터를 기다리는지 확인해야겠다고 생각했다.

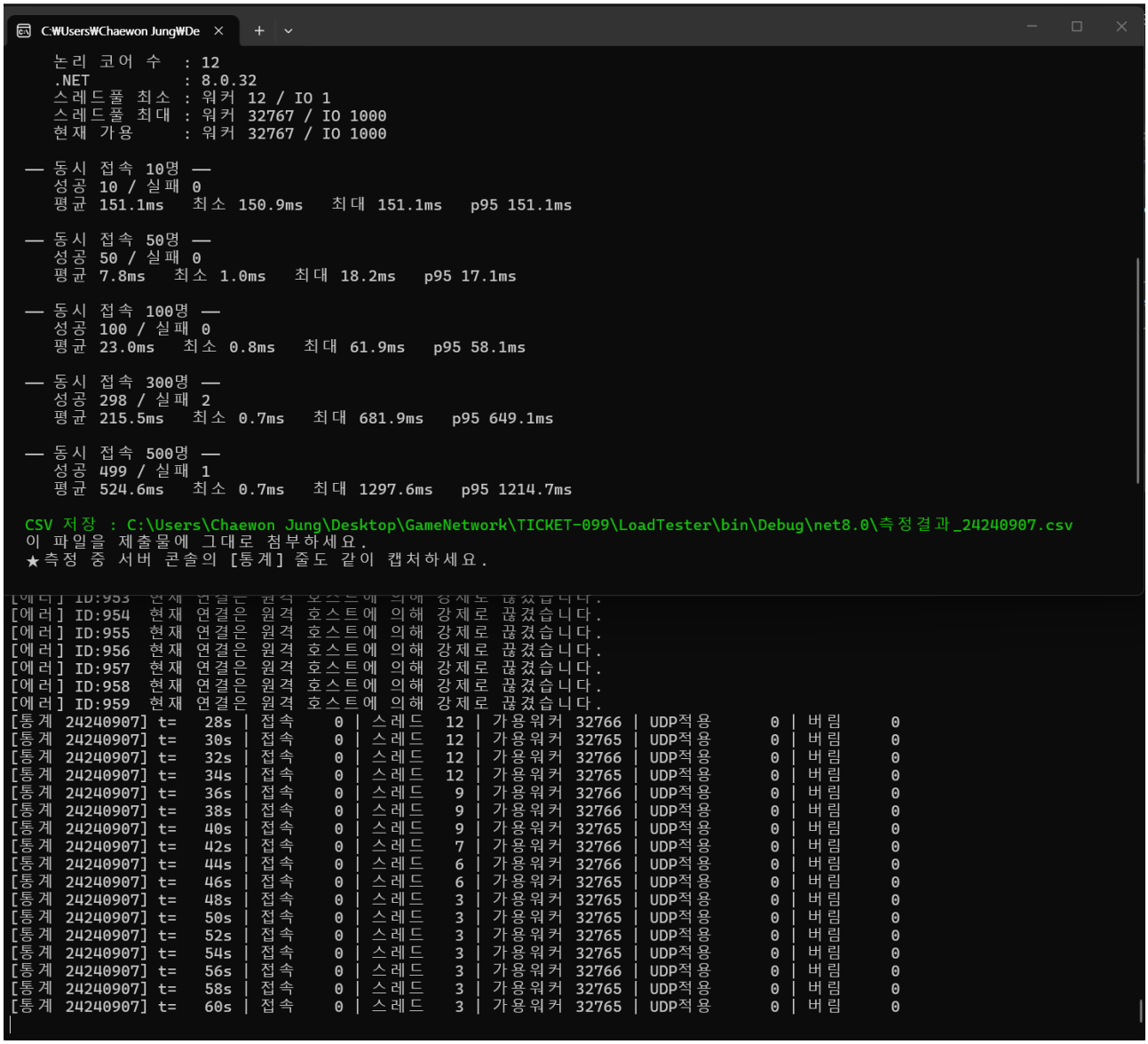


그림 28. D-3 수정 후 — 제출용 13:56 동시 접속 측정 결과

증거: B_수정전_서버통계.png, B_수정후_서버통계.png, B_수정후_측정중통계.png, 측정결과_24240907.csv

최종 판단

원본 외주 서버는 일부 기능이 정상적으로 동작했지만, 여러 오류가 확인되어 수정 없이 사용하기는 어렵다고 판단했다. TCP 분할 수신과 비정상 패킷 처리에서는 기능 오류가 있었고, UDP 의 중복 번호·데이터 길이·주소 검증과 클라이언트 수신, 퇴장 알림에서도 수정할 부분이 확인됐다. 원본 서버는 동시 접속 50 명 구간부터 실패가 크게 늘어 업체의 안정성 주장과도 맞지 않았다.

수정 후에는 준비한 기능 테스트에서 확인된 문제가 개선됐고, 동시 접속 500 명 시험에서 499 명이 성공했다. 다만 이것은 내 PC 와 이번 테스트 조건에서 확인한 결과다. 다른 네트워크 환경이나 더 많은 접속에서도 문제가 없다고 보장할 수는 없다. 따라서 납품 원본은 즉시 인수하지 않고, 수정본을 기준으로 추가 검증하는 것이 적절하다고 생각한다.

증거 자료 목록

아래 파일은 최종 제출 ZIP 의 증거/ 폴더에 포함한다. 보고서 본문에는 수정 전후를 비교할 수 있는 대표 화면을 배치하고, 나머지 이미지는 원본 크기로 증거 폴더에서 확인할 수 있도록 한다.

구분	파일명	확인 내용
공통	01_서버_초기상태.png	학번과 TCP·UDP 포트 확인
공통	02_클라이언트_TCP 연결_UDP 수신오류.png	최초 TCP 연결 및 UDP 수신 오류
A-1	A_버그 1_수정전_서버오류.png	분할 수신 관련 서버 오류
A-1	A_버그 1_수정전_테스트결과.png	원본 분할 전송 결과
A-1	A_버그 1_수정후.png	수정 후 분할 전송 결과
A-2	A_버그 2_수정전.png	Type 누락 패킷과 서버 예외
A-2	A_버그 2_수정후.png	서버 유지 및 경고 확인
A-3	A_버그 3_수정전.png	UDP 적용 21·버림 14
A-3	A_버그 3_수정후.png	UDP 적용 20·버림 15
A-3	A_버그 3_마지막위치확인.png	마지막 위치 확인
A-4	A_버그 4_수정전.png	재접속 과정의 위치 수신 오류
A-4	A_버그 4_수정후.png	수정 후 수신 데이터 확인
A-5	A_버그 5_수정전.png	주소 불일치 패킷이 적용 횟수에 포함됨
A-5	A_버그 5_수정후.png	주소 불일치 경고와 적용 차단
A-6	A_버그 6_수정전_초기상태.png	UDP 수신 수정 전 클라이언트

구분	파일명	확인 내용
A-6	A_버그 6_수정전_기능테스트.png	Bind 오류 재현
A-6	A_버그 6_수정후.png	Bind 오류 해결 확인
A-7	A_버그 7_수정전_퇴장알림누락.png	퇴장 알림 누락
A-7	A_버그 7_수정후_퇴장알림표시.png	퇴장 알림 표시
추가 확인	A_연속전송_정상검증.png	채팅 연속 전송 정상 검증
B	B_수정전_PC 정보.png	PC 환경 정보
B	B_수정전_서버통계.png	수정 전 부하 시험과 서버 통계
B	B_수정후_서버통계.png	수정 후 서버 통계
B	B_수정후_측정중통계.png	추가 측정 중 서버 통계
C	C_의견 3_기존코드_동시접속정상.png	업체 의견 3 번 기존 코드 실험
C	C_의견 3_await 적용_동시접속검증.png	await 적용 시 접속 지연 실험
D	D_버그 3_임시검증코드.png	UDP 시퀀스 처리 점검 당시 임시 코드
D	D_최종_커밋기록.png	Git 커밋 기록